INTERVIEW PREPARATION GUIDE

GCC Java Backend Developer NCG

Spring Boot · Microservices · Kafka · Docker · Kubernetes · BFSI
Prepared for Neelima Jana · Interview: Monday 16 November

How to use this guide. Work the sections in order. The plan on the next page is timed — if you fall behind, drop from the bottom, never from the top.

● **Red dot** = high priority, almost certainly asked. ●● = your differentiator. ● **Amber** = worth knowing.

✓ = self-check before moving on. ⚠ = common trap or something to verify.

Contents

Plan	NCG — GCC Java Backend Developer (BFSI)	3
01	Core Java and Collections	9
02	Java 8 and Functional Programming	20
03	Spring Boot	27
04	JPA, Hibernate and the N+1 Problem	37
05	Microservices	45
06	Kafka and RabbitMQ	52
07	Spring Security and JWT	57
08	SQL and Query Optimisation	64
09	Docker, Kubernetes and Containerisation	71
10	CI/CD and DevOps	78
11	Coding Round	82
12	Backend System Design	89
13	Talking About Your Experience	93
14	HR, Managerial and the Offer Conversation	98
15	Delivery, Code Review and Working With Clients	103
16	Final Cheat Sheet — INTERVIEW MORNING ONLY	110

NCG — GCC Java Backend Developer (BFSI)

Neelima Jana · Interview: Monday 16 November · Hyderabad · Job ID 88875

How this guide is written

Every topic follows the same four steps:

1. **The context** — what it is and what problem it solves, in plain words.
2. **A simple example** — small code you can follow.
3. **Why it matters** — where it is used in real work.
4. **"Say this"** — the answer to give in the room.

Every question has a written answer in the words you would say, and most carry a **real-world example** anchored to the banking platform you actually worked on. Learn the shape, not the wording.

1. A note on who you are interviewing with

The job description is from **NCG (NetConnect Global)** — a digital engineering and professional services company, Hyderabad, hiring for a **GCC** role in **BFSI**.

You mentioned IBM. **GCC** means **Global Capability Centre** — the in-house engineering centre a large enterprise runs in India. NCG is a services company that staffs those centres, so the most likely arrangement is **NCG is the employer and the GCC is the client**.

> ● **Confirm this with the recruiter before Monday — it is a completely normal question and**
> **it changes things that matter to you:**
> - Who is on the payroll — NCG, or the client directly?
> - Is this a deputation to a client site, and is there a path to conversion?
> - Which client, and which platform would you be on?
>
> *"Could you tell me a bit about the engagement — would I be on NCG payroll working at the client's GCC, and which client is it?"* Ask it early, in the HR or screening conversation.

What is certain from the JD either way: **Java, Spring Boot, microservices, BFSI, Hyderabad, client-facing, reporting to a technical lead.** That is what you prepare for.

2. The JD, decoded against your resume

The posting lists a lot of technologies. Here is what you actually have, and what you do not.

✓ Where you are strong — these are the bulk of the interview

The JD asks	You	File
Core Java 8+	Java 8+, OOP, collections, exceptions	`01` , `02`
Spring Boot, Spring MVC, Spring Data JPA	All three, in production	`03`
Hibernate / JPA	Yes — and you tuned it	`04`
RESTful APIs	Yes, with exception handling	`03` , `12`
Microservices architecture	The Classifieds platform — Eureka, API Gateway, 3 services	`05`
Kafka, RabbitMQ	**Both, in production, for different reasons**	`06`
JWT, application security	Spring Security RBAC, JWT	`07`
MySQL, query optimisation, indexing	Yes, with real numbers	`08`
MongoDB / NoSQL	On your resume	`08`
Git, Maven	Yes	`15`
Performance tuning, profiling	**15s → 6s and 2–3s → under 1s**	`04` , `13`
Production troubleshooting, root cause analysis	Yes — execution plans and logs	`13` , `15`
BFSI domain	**A UK private bank. This is a real advantage.**	throughout

⚠ Where you have gaps — four of them, all manageable

The JD asks	Your position	File
Docker, Kubernetes, containerisation	**Not on your resume at all.** Named three times in the JD	`09` ●●
Jenkins / GitHub Actions / Azure DevOps	You worked *inside* a SIT/UAT release process, did not own a pipeline	`10` ●
OAuth2	You have JWT and Spring Security, not a full OAuth2 integration	`07` Q6b
PostgreSQL, Oracle, ActiveMQ, Gradle, Java 11/17	Adjacent to what you have — dialect and syntax, not new concepts	`08` , `06` , `15` , `02`

● The pattern for every gap — use it all day

Name the boundary first, then show the understanding.

> *"I have not owned the container and deployment side — that was a separate team on our platform. So I would call that understanding rather than hands-on. What I do know is the model and why it is used..."*

Then explain it properly. **At two years that is a strong answer.** Claiming hands-on Kubernetes and then being asked why a pod is in CrashLoopBackOff is not — and the second question always comes.

Docker and Kubernetes are the one gap worth genuinely closing this weekend. You have three
days, it is three hours of reading, and it is named three times in the spec. File 09.

● On the experience band

The posting says **Level 3 (3–6 years)**. Your engineering experience is **December 2024 to June 2026 — about 18 months** — plus the recruitment year, so roughly **two years four months professional**.

You are under the band. You already have the interview, so someone looked at your profile and still wanted to talk; ranges are sourcing guidance, not a gate. But **answer with dates and never round up** — these roles verify against payslips, and your band and offer are set from the verified number. The full answer is in **file 13, section 5**.

---

3. What the interview will probably look like

Round 1 — Technical (45–60 min). Core Java, collections, Spring Boot, JPA, SQL, usually a coding question. → Files 01–04, 08, 11.

Round 2 — Technical / architecture (45–60 min). Microservices, messaging, security, containers and CI/CD, plus a deep dive into what you built. → Files 05, 06, 07, 09, 10, 12, 13.

Round 3 — Managerial / HR (20–30 min). Client-facing fit, the gap, availability, salary.
→ Files 14, 15.

> ⚠ This is my prediction from how these roles usually run, not something I can confirm. Ask the recruiter what the rounds are — it costs nothing and they usually tell you.

The BFSI instinct to carry into every answer: whenever you describe a design or a fix, add the sentence most candidates never say — **"and here is how we would know afterwards what happened."** Audit trail, traceable logging, idempotency so a payment is never taken twice, authorisation enforced server side, no sensitive data in logs. You worked on a bank's systems; that instinct is yours to show.

---

4. The three-day plan — Friday evening to Sunday

Friday evening (tonight) — close the biggest gap

Block	Topic	File
0:15	Read this file	`README.md`
1:30	**Docker and Kubernetes** ●● — your largest gap, named three times	`09-docker-kubernetes.md`
0:45	**CI/CD and DevOps** ●	`10-cicd-and-devops.md`
0:30	**Your experience** — the gap answer and the band answer, out loud	`13-your-experience.md` §5

Saturday — the core, where most of the interview lives

Block	Topic	File
1:45	**Core Java and Collections** ●● — HashMap above all	`01-core-java.md`
1:30	**Spring Boot** ●●	`03-spring-boot.md`
1:15	**JPA, Hibernate, N+1** ●● — your own story	`04-jpa-hibernate.md`
1:00	**SQL, indexing, Oracle/Postgres/Mongo** ●	`08-sql-and-databases.md`
0:45	**Java 8, and what 11 and 17 added** ●	`02-java8-functional.md`
Evening	**Your four stories, said out loud** ●●	`13-your-experience.md`

Sunday — architecture, security and rehearsal

Block	Topic	File
1:15	**Microservices** ●	`05-microservices.md`
1:00	**Kafka, RabbitMQ, ActiveMQ** ● — your differentiator	`06-kafka-rabbitmq.md`
0:45	**Spring Security, JWT and OAuth2** ●	`07-spring-security-jwt.md`
1:00	**Coding round** — type the problems, do not read them	`11-coding-round.md`
0:45	**Delivery, code review, Maven/Gradle**	`15-delivery-and-code-review.md`
0:45	**HR and managerial** — say these out loud	`14-hr-and-behavioural.md`
0:30	Skim: system design	`12-system-design.md`
By 11pm	**Sleep.** Do not cram Sunday night.	—

Monday morning

45 minutes on `16-final-cheatsheet.md` and nothing else. Then say out loud, twice: your intro, the gap answer, and the 15s → 6s story. **Learn nothing new.**

If you fall behind

Drop in this order: system design → coding round → delivery → Java 11/17 detail.
Never drop: Core Java, Spring Boot, JPA, file 09 (Docker/Kubernetes), or file 13.

---

5. The files

File	Topic	Priority
`01-core-java.md`	OOP, collections, HashMap internals, exceptions	● Highest
`02-java8-functional.md`	Streams, lambdas, Optional, **Java 11 and 17**	● High
`03-spring-boot.md`	IoC, DI, annotations, bean scopes, Spring MVC	● Highest
`04-jpa-hibernate.md`	JPA, lazy vs eager, **N+1**, transactions	● Highest
`05-microservices.md`	Patterns, Eureka, gateway, resilience, saga	● High
`06-kafka-rabbitmq.md`	Kafka vs RabbitMQ vs **ActiveMQ** — your edge	● High
`07-spring-security-jwt.md`	Spring Security, JWT, **OAuth2 and OIDC**	● High
`08-sql-and-databases.md`	Indexing, EXPLAIN, **Oracle/Postgres/Mongo**	● High
`09-docker-kubernetes.md`	**Containers, images, pods, services, probes**	●● **Biggest gap**
`10-cicd-and-devops.md`	**Pipelines, Jenkins/Actions/Azure, deployment**	● High
`11-coding-round.md`	Coding problems with full solutions	● Medium
`12-system-design.md`	Design framework and worked examples	● Medium
`13-your-experience.md`	**Your stories, the gap, the experience band**	● Highest
`14-hr-and-behavioural.md`	HR round, salary, questions to ask	● High
`15-delivery-and-code-review.md`	Code review, estimation, **Maven/Gradle**	● High
`16-final-cheatsheet.md`	One page. Monday morning only.	● Read last

---

6. Three rules for the room

1. Name your boundary before they find it. You are two years in, applying to a 3–6 band, with four technology gaps. Saying *"I have not owned that — here is what I do understand"* first makes everything else you say trustworthy. Bluffing collapses on the follow-up.

2. Give the why, not just the what. "HashMap stores key-value pairs" is a textbook answer. "It gives O(1) lookup because it hashes the key to a bucket instead of scanning" is an engineer's. Same length, completely different impression.

3. Bring every answer back to something you did. You have a UK bank, real performance numbers, Kafka and RabbitMQ used for different reasons, and production incidents you root-caused. That is more concrete evidence than most candidates with four years. Use it.

01 · Core Java and Collections

Time needed: 105 minutes

This is where Infosys screens hardest. Service company interviews go deep on fundamentals, because fundamentals are what transfer across the many different client projects they will put you on. Expect rapid-fire questions here.

If you only master one topic in this file, make it HashMap. It is the single most asked Java interview question in India.

Part A · OOP — the four pillars

You will be asked to name them and give an example of each. Have a real example ready, not a "Dog extends Animal" one.

Q1. Encapsulation

What it is: keeping data private and exposing it only through methods.

```
public class Account {
    private double balance;                        // nobody can touch this directly

    public void deposit(double amount) {
        if (amount <= 0) throw new IllegalArgumentException("Amount must be positive");
        this.balance += amount;                    // the rule is enforced here
    }

    public double getBalance() { return balance; }
}
```

Why it matters: if `balance` were public, any code anywhere could set it to a negative number and no validation would run. Encapsulation means the object protects its own rules. In a banking system that is not a style preference.

Say this.

> "Encapsulation means the object owns its data and its rules. The fields are private and the only way in is through methods that validate. In the card system, a credit limit could not just be assigned — it had to go through a method that checked the approval status and the band the customer qualified for. If that field were public, any code anywhere could set it and skip every check, and you would never find out where it happened."

Q2. Inheritance

One class reuses another's behaviour with `extends`. A `SavingsAccount extends Account` gets `deposit()` for free and adds interest calculation.

The follow-up: *"Why does Java not support multiple inheritance of classes?"*

> "Because of the diamond problem. If B and C both extend A and override the same method, and D extends both, the compiler cannot decide which version D inherits. Java avoids that by allowing only one superclass. You can implement multiple interfaces, and since Java 8 interfaces can have default methods — so if two interfaces give the same default method, the compiler forces you to override it and resolve the conflict explicitly."

Q3. Polymorphism ●

"One interface, many implementations." Two kinds, and interviewers want both.

	Overloading (compile-time)	Overriding (runtime)
What changes	Same method name, **different parameters**	Same signature, **different class**
Decided	At compile time, by the compiler	At runtime, by the actual object
Also called	Static polymorphism	Dynamic polymorphism

```
// Overloading - same name, different parameter lists
void notify(String email) { }
void notify(String email, String sms) { }

// Overriding - the subclass replaces the parent's behaviour
class Notifier      { void send() { System.out.println("generic"); } }
class EmailNotifier extends Notifier { @Override void send() { System.out.println("email"); } }

Notifier n = new EmailNotifier();
n.send();     // "email" - decided by the OBJECT, not the reference type
```

That last line is the whole point of runtime polymorphism, and it is what makes Spring's dependency injection work: the field is declared as an interface, and the concrete implementation is chosen at runtime.

Real-time example. In the notification flow, the service holds a `List<Notifier>` containing an email notifier and an SMS notifier. It calls `notifier.send(...)` on each without knowing or caring which is which. When a push notifier is added later, the sending code does not change at all — that is polymorphism doing real work.

Trap: *"Can you override a static method?"* → No. Static methods belong to the class, not the instance, so they are **hidden**, not overridden. Which one runs is decided by the reference type, not the object.

Say this.

> "Overloading is compile time — same method name, different parameters, and the compiler picks. Overriding is runtime — a subclass replaces the parent's behaviour, and which one runs depends on the actual object, not the reference type.
>
> Overriding is what makes dependency injection work. My service holds a `Notifier` reference, and at runtime it is an email or SMS implementation. The service never changes when a new channel is added."

Q4. Abstraction

Hiding *how* something works and exposing *what* it does.

● Abstract class vs interface — near-certain question

	Abstract class	Interface
Fields	Can have normal instance fields	Only `public static final` constants
Constructor	Yes	No
Methods	Abstract and concrete	Abstract, plus `default` and `static` since Java 8
Inheritance	One only	Many
Access modifiers	Any	Methods are public

When to use which — say it this way:

> "An abstract class is for an 'is-a' relationship where the subclasses share state and some common implementation — a base `Transaction` class holding amount and timestamp. An interface is a contract about capability with no shared state, like `Auditable` or `PaymentProcessor`. Since I can implement many interfaces but extend only one class, I default to interfaces and use an abstract class when there is real shared code and fields."

Real-time example — say this if they ask for one.

> "In the card system, `PaymentProcessor` was an **interface** with implementations for different rails — NEFT, RTGS, IMPS. Each one processes a payment completely differently, so there is no shared code to inherit, only a shared contract. The service just depends on `PaymentProcessor` and Spring injects the right one.
>
> `BaseTransaction` was an **abstract class**, because every transaction genuinely has an amount, a currency, a timestamp and an audit trail, plus a common `validate()` that all of them run. That is shared state and shared code, which an interface cannot give me."

Part B · Collections ●●

Q5. The map of the framework

```
Collection (interface)
├── List    – ordered, allows duplicates      → ArrayList, LinkedList, Vector
├── Set     – no duplicates                   → HashSet, LinkedHashSet, TreeSet
└── Queue   – FIFO processing                 → PriorityQueue, ArrayDeque

Map (separate, NOT a Collection) – key/value  → HashMap, LinkedHashMap, TreeMap, Hashtable
```

⚠ `Map` does **not** extend `Collection`. It stores pairs, not single elements. A small detail interviewers like to check.

Say this — how you choose a collection.

> "I pick by the access pattern. If I need order and index access, `List`, and `ArrayList` unless I have a reason. If I need uniqueness, `Set`. If I need lookup by a key, `Map`. Then the second question is whether ordering matters — `LinkedHashSet` or `TreeSet` instead of `HashSet` — and whether it is accessed by multiple threads, which means `ConcurrentHashMap` rather than `HashMap`."

Real-time example. In the recall batch: a `List` for the page of transactions being processed, a `Set` of already-processed reference numbers so a retry does not double process, and a `Map` of status code to description loaded once for the report.

Q6. ArrayList vs LinkedList ●

	ArrayList	LinkedList
Structure	Growable array	Doubly linked list
Get by index	**O(1)**	O(n) — must walk the chain
Add/remove at end	O(1) amortised	O(1)
Add/remove in middle	O(n) — shifts elements	O(1) *once you are at the node*
Memory	Less	More — each node stores two pointers

Say this:

> "I use ArrayList by default. Random access is O(1) and the elements sit together in memory, which is cache friendly. LinkedList only wins when I am inserting or removing frequently in the middle while already holding a reference to that position. In practice that is rare, and even for a queue I would use ArrayDeque."

Real-time example. A recall report loads a page of transactions and then iterates them to build the output — read-heavy, index access, no insertion in the middle. That is an `ArrayList`. I have not had a case in these systems where `LinkedList` was the right answer.

How does ArrayList grow? It starts at capacity 10, and when full it creates a new array about **1.5 times** the size and copies everything across. That copy is why `new ArrayList<>(expectedSize)` matters when you know the size up front.

Q7. ●● How does HashMap work internally?

Learn this properly. It is the most asked Java question in Indian interviews.

Storing a value — `map.put(key, value)`

1. Java calls `key.hashCode()` to get an int.
2. HashMap applies its own **hash spreading** on top: `h ^ (h >>> 16)`. This mixes the high bits into the low bits, because the next step only uses the low bits.
3. It finds the bucket: `index = (n - 1) & hash`, where `n` is the array length. Since `n` is always a power of two, this is a fast bitwise AND instead of a modulus.
4. If the bucket is empty, it stores the entry there.
5. If the bucket already has entries — a **collision** — it walks them comparing with `equals()`. Same key means replace the value; otherwise append the new entry.

The structure

Internally it is an **array of buckets**, and each bucket holds a **linked list** of entries. Since **Java 8**, once a single bucket exceeds **8 entries** (and the table is at least 64 long) that list is converted into a **balanced red-black tree**, turning worst-case lookup from O(n) into O(log n). Below 6 entries it converts back.

Resizing

Default capacity **16**, load factor **0.75**. When size passes 16 × 0.75 = **12**, the array doubles to 32 and everything is **rehashed** into new buckets. Resizing is expensive, which is why you presize a map you know will be large.

Complexity

O(1) average for get and put. O(log n) worst case since Java 8, thanks to treeification.

Real-time example — where you actually used one

> "Two places. In the report generation I loaded the reference data once — branch codes and status descriptions — into a `HashMap` keyed by code, so the loop could look each one up in O(1) instead of hitting the database per row. And on the Kafka consumer side, a map of processed message IDs is how you make a consumer idempotent: check the map before acting, so a redelivered message is a no-op."

● The follow-up that catches people: equals and hashCode

> "If you use a custom object as a key you must override both `hashCode()` and `equals()`. The contract is that **equal objects must have equal hash codes**. If I override `equals` but not `hashCode`, two objects that are equal can produce different hash codes, land in different buckets, and the map will never find the entry I stored. That is also why HashMap keys should be **immutable** — if a field used in the hash changes after insertion, the entry is effectively lost, because it is sitting in a bucket that no longer matches its hash."

Q8. HashMap vs Hashtable vs ConcurrentHashMap ●

	HashMap	Hashtable	ConcurrentHashMap
Thread safe	No	Yes	Yes
How	—	Locks the **whole map**	Locks only the affected **bucket**
Null key/value	One null key, many null values	Neither	Neither
Performance	Fastest single-threaded	Slow — everything serialises	Fast under concurrency
Status	Standard	**Legacy, do not use**	Standard for concurrent use

> "Hashtable synchronises every method, so all threads queue behind one lock. That is why it is effectively obsolete. ConcurrentHashMap locks at bucket level, so threads writing to different buckets do not block each other, which is why it scales."

Real-time example. A Spring `@Service` is a **singleton**, so any collection held in a field is shared by every request thread at once. If I keep an in-memory counter or cache in a service — say a per-branch request count — a plain `HashMap` will corrupt under load and can even spin forever on an older JDK. That field has to be a `ConcurrentHashMap`.

Q9. HashSet vs LinkedHashSet vs TreeSet

	Ordering	Backing structure	Complexity
HashSet	None guaranteed	HashMap	O(1)
LinkedHashSet	**Insertion order**	HashMap + linked list	O(1)
TreeSet	**Sorted**	Red-black tree	O(log n)

Useful detail: `HashSet` is literally a `HashMap` where every value is the same dummy object. That is why its rules about `hashCode` and `equals` are identical.

Real-time example. `HashSet` for "which account numbers have already been processed in this batch" — I only care about membership. `TreeSet` when the order matters, like interest rate slabs that must be walked from lowest to highest to find the applicable band. `LinkedHashSet` when I need uniqueness but the output must stay in the order the records arrived, which matters for a report a human reads.

Say this.

> "All three guarantee uniqueness; the difference is ordering and cost. `HashSet` is O(1) with no order guarantee. `LinkedHashSet` is also O(1) but keeps insertion order, which matters when a human reads the output. `TreeSet` keeps them sorted, which costs O(log n) because it is a red-black tree. I use `HashSet` by default and only pay for ordering when I actually need it."

Q10. Fail-fast vs fail-safe iterators

> "Fail-fast iterators, like ArrayList's and HashMap's, throw `ConcurrentModificationException` if the collection is structurally modified while iterating. They track a `modCount` and check it on each `next()`. Fail-safe iterators, like ConcurrentHashMap's or CopyOnWriteArrayList's, work on a snapshot or allow concurrent access, so they do not throw — but they may not reflect the very latest changes."

How do you remove safely while iterating? Use `iterator.remove()`, or `collection.removeIf(predicate)`. Never `list.remove()` inside a for-each loop.

Q11. Comparable vs Comparator •

```
// Comparable - the class defines its OWN natural ordering. One only.
class Txn implements Comparable<Txn> {
    public int compareTo(Txn other) { return this.date.compareTo(other.date); }
}

// Comparator - ordering defined OUTSIDE the class. As many as you like.
Comparator<Txn> byAmount = Comparator.comparing(Txn::getAmount);
list.sort(byAmount.reversed());
```

> "Comparable is the natural order and lives inside the class, so there can be only one. Comparator lives outside, so I can have several — sort transactions by date on one screen and by amount on another without touching the entity."

Real-time example. `Transaction` implements `Comparable` on `createdAt`, because newest-first is its natural order everywhere in the application. But the recall report

screen lets the user sort by amount, by status or by customer name — three `Comparator`s, none of which required changing the entity.

Part C · Strings, immutability and memory

Q12. Why is String immutable? ●

Once created, a String's value can never change. `s.concat("x")` returns a **new** String.

The four reasons — give at least two:

1. **String pool.** Java reuses identical literals to save memory. That is only safe if nobody can change them underneath other references.
2. **Security.** Usernames, file paths and DB URLs are Strings. If they were mutable, a value could be validated and then changed before use.
3. **Thread safety.** Immutable objects can be shared across threads with no locking.
4. **Hashcode caching.** String caches its hash, which makes it a fast HashMap key. That is only valid if the value cannot change.

Real-time example. Account numbers and transaction reference numbers are Strings used as map keys and passed between services. If a String were mutable, code holding a reference could change an account number *after* it had been validated and after it had been used to place an entry in a map — the entry would then be unreachable. Immutability is what makes them safe to share.

Say this.

> "Because too much depends on a String not changing under you. The string pool reuses identical literals, which is only safe if nobody can modify one. Strings are used for account numbers, file paths and connection URLs, so if they were mutable a value could be validated and then changed before it was used. And String caches its own hash code, which is what makes it a fast HashMap key — that cache is only valid because the value is fixed."

Q13. String vs StringBuilder vs StringBuffer

	Mutable?	Thread safe	Use when
String	No	Yes (immutable)	Fixed values
StringBuilder	Yes	**No**	Building strings — the default
StringBuffer	Yes	Yes (synchronised)	Legacy; rarely needed

> "Concatenating in a loop with `+` creates a new String every iteration, so a thousand-iteration loop creates a thousand throwaway objects. StringBuilder mutates one buffer, so it is O(n) instead of O(n²)."

Q14. `==` vs `.equals()`

`==` compares **references** — are these the same object in memory.
`.equals()` compares **values** — as defined by the class.

```
String a = "hello";                 // goes into the string pool
String b = "hello";                 // reuses the SAME pooled object
String c = new String("hello");     // forces a NEW object on the heap

a == b        // true  - same pooled reference
a == c        // false - different objects
a.equals(c)   // true  - same characters
```

This is a favourite quick question. Learn the three lines.

Say this.

> " `==` compares references — is this the same object in memory. `.equals()` compares values, as the class defines them. For Strings this catches people out because literals are pooled, so two identical literals are the same object and `==` happens to be true. But `new String("abc")` forces a separate object and `==` is false, while `.equals()` is still true. So I never use `==` for value comparison — it works by accident until it does not."

Q15. Stack vs Heap

- **Stack** — one per thread. Holds method calls, local variables and references. Freed automatically when the method returns.
- **Heap** — shared by all threads. Holds all objects. Managed by the garbage collector.

`StackOverflowError` means runaway recursion. `OutOfMemoryError` means the heap is full, usually a leak or an unbounded collection.

Say this.

> "The stack is per thread and holds method frames, local variables and references. It is cleaned up automatically when a method returns. The heap is shared across threads and holds all the objects, and the garbage collector manages it. A `StackOverflowError` means runaway recursion; an `OutOfMemoryError` means the heap filled up, which in my experience usually means something unbounded — loading an entire table into a list instead of paginating it."

Part D · Exceptions

Q16. Checked vs unchecked ●

```
Throwable
├── Error              – JVM level, do not catch (OutOfMemoryError)
└── Exception
    ├── RuntimeException  – UNCHECKED (NullPointerException, IllegalArgumentException)
    └── everything else   – CHECKED (IOException, SQLException)
```

- **Checked**: the compiler forces you to catch it or declare `throws`. Represents recoverable, expected conditions — a missing file, a network failure.
- **Unchecked**: extends `RuntimeException`. Usually a programming bug. No compiler enforcement.

Say this:

> "In application code I mostly use unchecked exceptions and a custom hierarchy — a `TransactionNotFoundException` extending `RuntimeException` — because forcing every caller to catch and rethrow adds noise. I handle them centrally in Spring with `@ControllerAdvice` so the API returns a consistent error shape. It also matters for `@Transactional`, because by default Spring only rolls back on unchecked exceptions."

That last sentence connects two topics and sounds like real experience. Use it.

Real-time example. In the recall APIs I had `TransactionNotFoundException` and `InvalidRecallStateException`, both unchecked, both extending a common `BusinessException`. `@RestControllerAdvice` mapped them to 404 and 409 respectively, so the UI team always got the same error shape. Making them unchecked also mattered for `@Transactional`, because a checked exception would have **committed** the transaction instead of rolling it back.

Q17. try-with-resources and finally

```
try (Connection conn = dataSource.getConnection()) {
    // conn.close() is called automatically, even if an exception is thrown
}
```

Anything implementing `AutoCloseable` works. This replaced the old `finally { if (conn != null) conn.close(); }` pattern.

Trap: *"Does `finally` always run?"* → Almost always. Not if the JVM exits (`System.exit()`) or the thread is killed. And if `finally` contains a `return`, it overrides the one in `try` — which is why you never return from `finally`.

Say this.

> "Try-with-resources closes anything that implements `AutoCloseable` automatically, even if an exception is thrown, so I do not need a `finally` block that null-checks and closes. It replaced a pattern that people got wrong constantly — forgetting to close in one branch, or throwing a second exception inside `finally` that hid the original one. With JPA and Spring I rarely manage connections myself, but for file and stream handling it is what I use."

Q18. `throw` vs `throws` vs `final` vs `finally` vs `finalize`

- `throw` — actually raise an exception now.
- `throws` — declare that this method may raise one.
- `final` — a constant, a non-overridable method, or a non-extendable class.
- `finally` — the block that runs whatever happens.
- `finalize()` — a deprecated, unreliable method the GC used to call. Never use it.

Say this.

> " `throw` raises an exception now, `throws` declares that a method might. `final` means a constant, a method that cannot be overridden or a class that cannot be extended. `finally` is the block that runs whatever happens. And `finalize()` is a deprecated method the garbage collector used to call — it is unreliable and should not be used; try-with-resources or an explicit close is the correct approach."

✓ Check yourself before moving on

1. Explain **HashMap put and get end to end** — hashing, buckets, collisions, treeification, resizing. Out loud, without notes.
2. Explain the `equals` / `hashCode` contract and what breaks if you ignore it.
3. Overloading vs overriding, with an example of each.
4. Abstract class vs interface, and when you would pick each.
5. Explain the three lines of the `==` vs `.equals()` String example.

02 · Java 8 and Functional Programming

Time needed: 60 minutes

Your resume says "Java 8+", so this **will** be tested. Interviewers usually ask you to explain streams and then write one on the spot.

First, the context: what problem did Java 8 solve?

Before Java 8, processing a collection meant writing *how* to loop:

```
// imperative - you describe every step
List<String> names = new ArrayList<>();
for (Txn t : txns) {
    if (t.getAmount() > 1000) {
        names.add(t.getCustomerName().toUpperCase());
    }
}
Collections.sort(names);
```

Java 8 let you describe *what* you want instead:

```
// declarative - the intent is visible in one read
List<String> names = txns.stream()
        .filter(t -> t.getAmount() > 1000)
        .map(t -> t.getCustomerName().toUpperCase())
        .sorted()
        .collect(Collectors.toList());
```

Same result, but the second reads as a sentence: filter, map, sort, collect. That is the point — less boilerplate and clearer intent.

Q1. What is a lambda? ●

A short way to write an implementation of an interface that has exactly one method.

```
// before Java 8
Runnable r = new Runnable() {
    public void run() { System.out.println("running"); }
};

// with a lambda
Runnable r = () -> System.out.println("running");
```

The compiler knows `Runnable` has one method, so you only supply the body.

Say this.

> "A lambda is a compact way to supply the implementation of an interface that has one method. Before Java 8 I would write an anonymous inner class of five lines to pass behaviour into a method; a lambda is the same thing with the boilerplate removed. It is what makes the Streams API readable — `filter(t -> t.getAmount() > 1000)` says what it does in one line."

Q2. What is a functional interface? ●

An interface with **exactly one abstract method**. That is what makes it a valid target for a lambda. `@FunctionalInterface` is optional but makes the compiler enforce it.

The four you must know:

Interface	Method	Takes	Returns	Used by
`Predicate<T>`	`test(T)`	T	boolean	`filter`
`Function<T,R>`	`apply(T)`	T	R	`map`
`Consumer<T>`	`accept(T)`	T	void	`forEach`
`Supplier<T>`	`get()`	nothing	T	`orElseGet`, lazy values

```
Predicate<Txn> isLarge  = t -> t.getAmount() > 1000;
Function<Txn, String> name = t -> t.getCustomerName();
Consumer<Txn> log       = t -> System.out.println(t);
Supplier<Txn> empty     = () -> new Txn();
```

Also worth naming: `BiFunction<T,U,R>` (two arguments), `UnaryOperator<T>` (T to T), and `BinaryOperator<T>` (two Ts to one T, used by `reduce`).

Say this.

> "An interface with exactly one abstract method. That single method is what lets the compiler match a lambda to it. `@FunctionalInterface` is optional but I add it, because it makes the compiler fail if someone later adds a second abstract method and silently breaks every lambda using it. The four I use most are `Predicate` for filtering, `Function` for mapping, `Consumer` for side effects and `Supplier` for a lazily produced value."

Q3. Method references

Shorthand for a lambda that just calls one method.

```
.map(t -> t.getCustomerName())   →   .map(Txn::getCustomerName)
.forEach(x -> System.out.println(x))  →  .forEach(System.out::println)
```

Say this.

> "A method reference is a shorter form of a lambda that does nothing but call one method. `t -> t.getCustomerName()` becomes `Txn::getCustomerName`. It is purely readability — the compiler produces the same thing — but in a stream chain it removes noise and leaves only the intent."

Q4. ●● Streams — intermediate vs terminal operations

This distinction is asked constantly.

Intermediate operations return another Stream and are **lazy** — they do nothing until a terminal operation runs. `filter`, `map`, `flatMap`, `sorted`, `distinct`, `limit`, `skip`, `peek`.

Terminal operations produce a result and **trigger** the whole pipeline. `collect`, `forEach`, `reduce`, `count`, `min`, `max`, `anyMatch`, `allMatch`, `findFirst`.

```
txns.stream().filter(t -> t.getAmount() > 1000);   // NOTHING happens - no terminal op
```

Why laziness matters — the answer that impresses:

> "Because intermediate operations are lazy, the stream can fuse them into a single pass and stop early. `stream().filter(...).findFirst()` does not filter the whole list and then take the first — it stops at the first match. With `limit(10)` on a million records, only enough elements are processed to produce ten."

A stream can only be consumed once. Reusing one throws `IllegalStateException: stream has already been operated upon or closed`.

Q5. The stream methods you must be able to write ●

```java
// filter + map + collect
List<String> big = txns.stream()
        .filter(t -> t.getAmount() > 1000)
        .map(Txn::getCustomerName)
        .collect(Collectors.toList());

// sum with reduce, and the shorter specialised form
double total = txns.stream().mapToDouble(Txn::getAmount).sum();
double same  = txns.stream().map(Txn::getAmount).reduce(0.0, Double::sum);

// GROUP BY - very commonly asked
Map<String, List<Txn>> byStatus =
        txns.stream().collect(Collectors.groupingBy(Txn::getStatus));

// group and count
Map<String, Long> countByStatus =
        txns.stream().collect(Collectors.groupingBy(Txn::getStatus, Collectors.counting()));

// group and sum
Map<String, Double> totalByStatus = txns.stream()
        .collect(Collectors.groupingBy(Txn::getStatus,
                Collectors.summingDouble(Txn::getAmount)));

// sort by a field, descending
txns.stream().sorted(Comparator.comparing(Txn::getAmount).reversed())

// flatMap - flatten nested collections into one stream
List<Item> all = orders.stream()
        .flatMap(o -> o.getItems().stream())
        .collect(Collectors.toList());
```

Real-time example — where this replaced real code. The report summary needed totals per status: how many recalls are pending, approved and rejected, and the total value of each. That was originally nested loops with counters and a map being built by hand. One `groupingBy` with `summingDouble` replaced about twenty lines, and the intent is now visible in one read.

⚠ But note the boundary: **do this in SQL when the data is in the database.** Streaming a million rows into memory to group them is much slower than a `GROUP BY`. Streams are for data you already have in hand.

That caveat is worth saying out loud — it shows you know where the work should happen.

● **`map` vs `flatMap`** is a guaranteed question:

> " `map` transforms one element into one element. `flatMap` transforms one element into a **stream** of elements and then flattens them all into a single stream. If each order has a list of items, `map` gives me a stream of lists, and `flatMap` gives me a stream of items."

Real-time example. A recall request has a list of approvals. To get every approver across a page of requests, `map` would give me a stream of *lists* of approvals; `flatMap` gives me a single stream of approvals I can then filter and count.

Q6. Optional •

Introduced to make "this might be absent" explicit in the type, instead of returning `null` and hoping the caller checks.

```
Optional<Txn> found = repository.findById(id);

// good
String name = found.map(Txn::getCustomerName).orElse("Unknown");
found.ifPresent(t -> log.info("Found {}", t.getId()));
Txn txn = found.orElseThrow(() -> new TransactionNotFoundException(id));

// bad - this is just a null check with extra steps
if (found.isPresent()) { Txn t = found.get(); }
```

> "Optional makes the possibility of absence part of the method signature, so the caller cannot ignore it by accident. I use `map`, `orElse` and `orElseThrow` rather than `isPresent` and `get`, because that defeats the purpose. Spring Data returns `Optional` from `findById` for exactly this reason."

Real-time example. `txnRepository.findById(id)` returns `Optional<Transaction>`. In the recall service that becomes `.orElseThrow(() -> new TransactionNotFoundException(id))`, which the `@RestControllerAdvice` turns into a clean 404. Before `Optional`, that same code path returned `null` and produced a `NullPointerException` and a 500 — which tells the caller nothing useful.

`orElse` vs `orElseGet` — a good detail: `orElse(buildDefault())` evaluates the argument **always**, even when a value is present. `orElseGet(() -> buildDefault())` only runs it when needed. It matters when the default is expensive.

Q7. Parallel streams — and when not to use them

```
list.parallelStream().filter(...).collect(...);
```

> "It splits the work across the fork-join pool. It only helps with a large dataset and a genuinely CPU-bound operation with no shared mutable state. For small collections the coordination overhead makes it slower, and for I/O-bound work like database calls it is the wrong tool. I would measure before using it, not assume."

Q8. Other Java 8 features worth naming

- **Default and static methods in interfaces** — let you add a method to an interface without breaking every existing implementation.
- `java.time` — `LocalDate`, `LocalDateTime`, `Duration`. Immutable and thread safe, unlike the old `Date` and `Calendar`.
- `CompletableFuture` — asynchronous composition, useful for calling several services in parallel.

Say this.

> "The three I actually use beyond streams. **Default methods** in interfaces, which let a library add a method without breaking every existing implementation — that is how `Collection` gained `stream()` without breaking the world. `java.time`, because the old `Date` and `Calendar` were mutable and not thread safe, and `LocalDate` and `LocalDateTime` are neither. And `CompletableFuture` when I need to call two independent services in parallel instead of one after the other."

Q9. Later versions, in one line each

If asked "what have you used beyond Java 8":
- **Java 11** — `var` for local variables, `String.isBlank()`, `strip()`, `lines()`.
- **Java 17 (LTS)** — records, sealed classes, switch expressions, text blocks.
- A **record** is a compact immutable data carrier: `record TxnDto(String id, double amount) {}` generates the constructor, getters, `equals`, `hashCode` and `toString`. Very useful for DTOs.

⚠ **The JD says "Java 8/11/17+" and your resume says Java 8+.** So expect this. Naming what changed in each LTS is enough — nobody expects production experience with all three.

Java 11 (LTS)

```
var list = new ArrayList<String>();   // local variable type inference
"  hi  ".strip();                     // Unicode-aware trim()
"".isBlank();                         // blank, not just empty
"a\nb".lines();                       // stream of lines
Files.readString(path);               // read a file in one call
list.toArray(String[]::new);
```

Java 17 (LTS)

```java
// record - an immutable data carrier. Constructor, getters,
// equals, hashCode and toString are generated for you.
record RecallRequestDto(Long id, String reference, BigDecimal amount) { }

String sql = """
    SELECT id, status FROM recall_request
    WHERE status = ?
    """;                                          // text block

String label = switch (status) {                   // switch EXPRESSION - returns a value,
    case PENDING, IN_REVIEW -> "Awaiting action";//  no fall-through, must be exhaustive
    case APPROVED           -> "Approved";
    case REJECTED           -> "Rejected";
};

if (obj instanceof RecallRequest req && req.isPending()) {   // pattern matching, no cast
    req.approve();
}

sealed interface Event permits Approved, Rejected { }        // restricts who can extend
```

Say this — honestly.

> "My production work has been Java 8, which is what the banking platform ran on. I have kept up with the LTS releases: Java 11 added `var`, the String utilities and `Files.readString`, and Java 17 added records, text blocks, switch expressions, pattern matching for `instanceof` and sealed classes.
>
> Records are the one I would most want day to day — a DTO goes from forty lines of constructor, getters, `equals` and `hashCode` to one line, and it is immutable by default, which is exactly what you want for something crossing an API boundary."

Do not claim production experience with a version you have not used. Knowing what changed and why is enough at this level.

✓ Check yourself before moving on

1. Write a stream that groups transactions by status and sums the amount in each group. From memory.
2. Explain intermediate vs terminal, and why laziness lets a stream stop early.
3. Explain `map` vs `flatMap` in two sentences.
4. Name the four core functional interfaces and which stream method uses each.

03 · Spring Boot

Time needed: 90 minutes

"Java → Spring Boot" is listed as the **primary skill** on the job description. Along with Core Java, this is where most of your technical round will be spent.

First, the context: what problem does Spring solve?

The problem: tight coupling

Without Spring, an object creates the things it depends on:

```
public class TransactionService {
    private final EmailNotifier notifier = new EmailNotifier();   // hard-wired
}
```

Two problems. To switch to SMS you must edit this class. And in a unit test you cannot replace the notifier with a fake, so your test sends real emails.

The solution: Inversion of Control

The object stops creating its dependencies and **receives** them instead:

```
@Service
public class TransactionService {
    private final Notifier notifier;                    // an interface

    public TransactionService(Notifier notifier) {      // handed in from outside
        this.notifier = notifier;
    }
}
```

Now the class depends on an interface, not a concrete class. Something else decides which implementation to supply. That "something else" is the Spring container.

Inversion of Control (IoC) is the principle: control of creating objects is inverted, moving from your class to the framework.
Dependency Injection (DI) is how Spring implements it: it constructs objects and passes their dependencies in.

Say it in one line:

> "IoC is the principle that the framework, not my class, controls object creation and wiring. Dependency injection is the mechanism. The practical benefit is that my classes depend on interfaces, so I can swap implementations and inject mocks in tests without changing the class."

Real-time example. The card approval service depends on a `NotificationSender` interface. In production Spring injects the RabbitMQ implementation. In unit tests I pass a mock and assert that approval publishes exactly one message — no broker, no emails, and the test runs in milliseconds. That is only possible because the service never says `new`.

And Spring Boot on top of Spring

Plain Spring needed a lot of XML and manual configuration. Spring Boot adds:
- **Auto-configuration** — sees a database driver on the classpath and configures a DataSource for you.
- **Starter dependencies** — `spring-boot-starter-web` pulls in a compatible set of libraries so you are not resolving versions by hand.
- **Embedded server** — Tomcat is inside the JAR, so `java -jar app.jar` runs it. No separate application server to install.
- **Production features** — Actuator gives health checks and metrics endpoints.

> "Spring Boot is Spring with opinionated defaults. It removes the configuration work so you start with a running application and override only what you need."

Q1. ● The three types of injection, and which to use

```
// 1. CONSTRUCTOR injection - what you should use
@Service
public class TxnService {
    private final TxnRepository repo;
    public TxnService(TxnRepository repo) { this.repo = repo; }
}

// 2. FIELD injection - convenient, but avoid
@Autowired private TxnRepository repo;

// 3. SETTER injection - for genuinely optional dependencies
@Autowired public void setRepo(TxnRepository repo) { this.repo = repo; }
```

Why constructor injection — four reasons, give two or three:
1. The field can be `final`, so the object is **immutable** once built.
2. Dependencies **cannot be missing** — the object cannot be constructed without them.
3. **Testable without Spring** — `new TxnService(mockRepo)` just works. With field injection you need reflection or a Spring context.

4. It makes bad design **visible** — a constructor with eight parameters is obviously doing too much. Field injection hides that.

Since Spring 4.3, if a class has one constructor you do not even need `@Autowired`.

Say this.

> "I use constructor injection. Three reasons. The fields can be `final`, so the object is immutable once it is built. The dependencies cannot be missing, because you cannot construct the object without them — with field injection you find out at runtime with a null pointer. And it is testable without Spring at all: `new TxnService(mockRepo)` just works, whereas field injection needs reflection or a full context.
>
> There is a fourth benefit — it makes bad design visible. A constructor with eight parameters obviously does too much. Field injection hides that."

Q2. ● The annotations you must know

Annotation	What it does
`@SpringBootApplication`	The main class. Combines the next three.
`@Configuration`	This class defines beans
`@EnableAutoConfiguration`	Turn on Boot's auto-config
`@ComponentScan`	Scan this package and below for components
`@Component`	A generic Spring-managed bean
`@Service`	A `@Component` marking business logic
`@Repository`	A `@Component` for data access — **also translates DB exceptions** into Spring's `DataAccessException` hierarchy
`@Controller`	Returns view names
`@RestController`	`@Controller` + `@ResponseBody` — returns JSON
`@Autowired`	Inject a dependency
`@Qualifier("name")`	Choose between multiple beans of the same type
`@Primary`	Make one bean the default when several match
`@Value("${prop}")`	Inject a property from configuration
`@Bean`	Declare a bean from a method, usually for third-party classes
`@Transactional`	Wrap the method in a database transaction
`@ControllerAdvice`	Global exception handling across controllers

Trap: *"`@Component` vs `@Service` vs `@Repository` — is there a functional difference?"*

> "Technically `@Service` and `@Component` behave identically; the difference is intent, so the layer is obvious to a reader and to tooling. `@Repository` is the exception — it does real work, translating vendor-specific database exceptions into Spring's `DataAccessException` hierarchy, so my service layer is not coupled to a particular database's error codes."

Q3. REST annotations

```
@RestController
@RequestMapping("/api/transactions")
public class TxnController {

    @GetMapping("/{id}")
    public TxnDto get(@PathVariable Long id) { ... }

    @GetMapping
    public List<TxnDto> search(@RequestParam(defaultValue = "0") int page) { ... }

    @PostMapping
    @ResponseStatus(HttpStatus.CREATED)
    public TxnDto create(@Valid @RequestBody CreateTxnRequest req) { ... }

    @PutMapping("/{id}")
    public TxnDto replace(@PathVariable Long id, @RequestBody TxnDto dto) { ... }

    @DeleteMapping("/{id}")
    public ResponseEntity<Void> delete(@PathVariable Long id) { ... }
}
```

`@PathVariable` reads from the URL path. `@RequestParam` reads a query parameter. `@RequestBody` deserialises the JSON body. `@Valid` triggers Bean Validation.

Say this.

> "`@RestController` is `@Controller` plus `@ResponseBody`, so the return value is serialised to JSON rather than resolved as a view name. `@RequestMapping` at class level sets the base path. Then `@PathVariable` reads a value out of the URL path, `@RequestParam` reads a query parameter, and `@RequestBody` deserialises the JSON body into an object. I put `@Valid` on the request body so Bean Validation runs before the method is entered, which means invalid input never reaches my business logic."

Q4. ● Bean scopes

Scope	Meaning
singleton	**The default.** One instance for the whole application context.
prototype	A new instance every time it is requested.
request	One per HTTP request (web only).
session	One per HTTP session (web only).

● **The follow-up:** *"Singleton beans are shared across all requests. Is that a problem?"*

> "Only if the bean holds mutable state. Spring beans should be stateless — request data belongs in method parameters and local variables, not in instance fields. A singleton service with a mutable instance field is a genuine concurrency bug, because many threads share that one object."

Real-time example — a real bug this causes. If a service stores the current request's user in an instance field to avoid passing it around, two concurrent approvals will overwrite each other's value, and one approval gets attributed to the wrong approver. In a bank that is an audit failure, not just a bug. Request-scoped data belongs in method parameters, or in the `SecurityContext`, never in a field on a singleton.

That answer shows you understand *why* the default is safe, which is what they are checking.

Q5. ● Global exception handling

Nearly always asked, because it separates people who have shipped an API from people who have followed a tutorial.

```java
@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(TransactionNotFoundException.class)
    public ResponseEntity<ErrorResponse> handleNotFound(TransactionNotFoundException ex) {
        return ResponseEntity.status(HttpStatus.NOT_FOUND)
                .body(new ErrorResponse("TXN_NOT_FOUND", ex.getMessage()));
    }

    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ResponseEntity<ErrorResponse> handleValidation(MethodArgumentNotValidException ex) {
        String msg = ex.getBindingResult().getFieldErrors().stream()
                .map(e -> e.getField() + ": " + e.getDefaultMessage())
                .collect(Collectors.joining(", "));
        return ResponseEntity.badRequest().body(new ErrorResponse("VALIDATION_ERROR", msg));
    }

    @ExceptionHandler(Exception.class)
    public ResponseEntity<ErrorResponse> handleAll(Exception ex) {
        log.error("Unexpected error", ex);      // log the detail...
        return ResponseEntity.status(500)
                .body(new ErrorResponse("INTERNAL_ERROR", "Something went wrong"));
                                                // ...but do not leak it to the client
    }
}
```

Say why it matters:

> "It keeps error handling out of every controller, so the API returns one consistent error shape. The last handler matters for security: I log the stack trace but return a generic message, because a raw stack trace tells an attacker about your framework versions and internal structure. On a banking system that is a real concern."

Real-time example. Before this was centralised, each controller had its own try/catch and every endpoint returned a slightly different error body — some with a `message`, some with an `error`, some just a stack trace. The UI team had to special-case each one. One `@RestControllerAdvice` gave every endpoint the same `{ code, message, timestamp }` shape, and their error handling became a single function.

Q6. Bean Validation

```
public class CreateTxnRequest {
    @NotBlank(message = "Account number is required")
    private String accountNumber;

    @NotNull @DecimalMin(value = "0.01", message = "Amount must be positive")
    private BigDecimal amount;

    @Email private String notifyEmail;
}
```

`@Valid` on the `@RequestBody` triggers it, and a failure raises `MethodArgumentNotValidException`, handled above.

⚠ Note `BigDecimal` for money, never `double`. Floating point cannot represent decimal fractions exactly, so `0.1 + 0.2` is not `0.3`. In a banking interview, using `BigDecimal` for currency is a detail that gets noticed.

Say this.

> "Bean Validation with `@Valid` moves input checking to the edge of the application, so the controller rejects bad requests before any business code runs. The rules live on the DTO next to the field, which is easier to keep correct than validation scattered through a service.
>
> The important caveat is that this is not a security control. It runs on my server, so it is trustworthy, but it is only one layer — for something like a transfer amount I would still check business rules in the service, because a validation annotation cannot know the account balance."

Real-time example. On the card request form the DTO carried `@NotBlank` on the employee ID and `@DecimalMin` on the limit. Those failures come back as a 400 with a field-level message, handled by the same `@RestControllerAdvice` as everything else, so the UI could show the error against the right field.

Q7. Configuration and profiles

```
**Say this.**
> "Profiles let one build behave differently per environment, so the artefact that passed
> SIT is the exact artefact that goes to UAT and production — only the profile changes.
> Anything environment-specific, especially credentials, comes from environment variables
> rather than being committed. And I use `@ConfigurationProperties` rather than scattering
> `@Value` annotations, because it groups related settings into one typed object that
> fails at startup if something is missing, instead of failing later at runtime."

# application.yml
spring:
  datasource:
    url: ${DB_URL}
    username: ${DB_USER}
    password: ${DB_PASSWORD}     # from environment, never committed
---
spring:
  config.activate.on-profile: dev
  jpa.show-sql: true
```

Run with `-Dspring.profiles.active=dev`. Profiles let one build behave differently per environment without rebuilding.

Real-time example — this is your SIT and UAT workflow. The same artefact is promoted through dev, SIT, UAT and production. Only the profile changes: different database URLs, different queue names, SQL logging on in dev and off in production. That is what makes it the *same build* being tested and released, rather than a rebuild per environment — which is the whole point of promoting a tested artefact.

```
@ConfigurationProperties(prefix = "app.recall")   // type-safe, grouped config
public class RecallProperties { private int maxDays; private String queueName; }
```

Q8. Spring Boot Actuator

Adds production endpoints: `/actuator/health` (used by load balancers and Kubernetes to decide if the instance is alive), `/actuator/metrics`, `/actuator/info`.
Say that you would **secure these**, because they expose internals.

Say this.

> "Actuator exposes operational endpoints — health, metrics and info. The health endpoint is the practical one: a load balancer or container orchestrator calls it to decide whether the instance should receive traffic, and it can include the database and message broker status, so an instance that has lost its database is taken out of rotation automatically.
>
> I would secure these endpoints, because they expose internals. On a banking system I would not have them open."

Q9. How does auto-configuration actually work?

A step above the basics, and a good one to know:

> "Spring Boot ships configuration classes annotated with conditions such as `@ConditionalOnClass` and `@ConditionalOnMissingBean`. At startup it evaluates those conditions against what is on the classpath and what you have already defined. So if H2 is on the classpath and I have not declared a DataSource, it creates one — but the moment I define my own bean, `@ConditionalOnMissingBean` backs off and mine wins. That is why defaults never fight your own configuration."

Q10. Testing a Spring Boot application

```
@WebMvcTest(TxnController.class)          // controller layer only, fast
class TxnControllerTest {
    @Autowired MockMvc mockMvc;
    @MockBean TxnService service;          // replaces the real bean

    @Test void returns404WhenMissing() throws Exception {
        when(service.findById(1L)).thenThrow(new TransactionNotFoundException(1L));
        mockMvc.perform(get("/api/transactions/1")).andExpect(status().isNotFound());
    }
}
```

- `@SpringBootTest` loads the whole context — integration tests, slower.
- `@WebMvcTest` / `@DataJpaTest` load one slice — much faster.
- `@MockBean` swaps a bean for a Mockito mock.

Your resume mentions unit tests for state-transition validation, so be ready to say what you actually tested and why.

Say this.

> "I write mostly slice tests rather than loading the whole context, because they are much faster and a fast suite is one people actually run. `@WebMvcTest` for the controller layer with the service mocked, `@DataJpaTest` for repository queries against an in-memory database, and plain JUnit with Mockito for service logic, which needs no Spring at all. `@SpringBootTest` I keep for a small number of genuine end-to-end paths.
>
> What I test first is the state transitions — that a rejected request cannot be approved, that an already-approved one cannot be approved twice. Those are the rules that cost money if they break."

✓ Check yourself before moving on

1. Explain IoC and DI, and why constructor injection is preferred — three reasons.
2. Explain what a singleton bean means for thread safety.
3. Write a `@RestControllerAdvice` handler from memory.
4. Explain how auto-configuration decides what to configure.

04 · JPA, Hibernate and the N+1 Problem

Time needed: 75 minutes

This file contains your best story. Both your performance achievements — 15s → 6s and 2–3s → under 1s — came from fixing N+1 queries. When the interviewer asks about Hibernate, you are not reciting theory, you are describing something you did. Make sure you can explain the mechanism precisely enough to prove that.

First, the context: what is JPA and what is Hibernate?

Your database stores **rows in tables**. Your Java code works with **objects**. Someone has to translate between them. Doing it by hand means endless JDBC boilerplate: open a connection, build SQL, loop the ResultSet, map each column to a field, close everything.

ORM (Object-Relational Mapping) automates that translation.

- **JPA** is the *specification* — a set of interfaces and annotations. It is a standard, not an implementation.
- **Hibernate** is the most common *implementation* of that specification.
- **Spring Data JPA** is a layer above both that generates repository implementations from method names, so you rarely write queries at all.

> "JPA is the specification, Hibernate is the implementation, and Spring Data JPA is the abstraction on top that generates the repository code. Coding against JPA interfaces means I could switch the provider without rewriting my entities."

Q1. Entity mapping basics

```java
@Entity
@Table(name = "transactions")
public class Transaction {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, length = 20)
    private String accountNumber;

    private BigDecimal amount;

    @Enumerated(EnumType.STRING)      // store "PENDING", not 0 - ordinals break on reorder
    private TxnStatus status;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "customer_id")
    private Customer customer;
}
```

⚠ `@Enumerated(EnumType.STRING)` is worth mentioning. The default is `ORDINAL`, which stores the enum's position as a number — so inserting a new value in the middle of the enum silently corrupts every existing row. In banking data that is a serious bug.

Say this.

> " `@Entity` marks the class as mapped to a table and `@Id` is the primary key, with `@GeneratedValue` telling Hibernate the database generates it. `@Column` constrains the mapping, and `@ManyToOne` with `@JoinColumn` maps the foreign key.
>
> Two details I always set explicitly. `@Enumerated(EnumType.STRING)` , because the default stores the enum's numeric position — so if someone inserts a new value in the middle of the enum, every existing row silently means something different. And `BigDecimal` for money, never `double` , because floating point cannot represent decimal fractions exactly and the rounding errors accumulate."

Q2. ● Lazy vs eager loading

- **EAGER** — the related entity is loaded immediately, with the parent.
- **LAZY** — a proxy is put in place, and the real data is only fetched when you first access it.

The defaults, which they may ask you to recite:

Relationship	Default
`@OneToOne`	EAGER
`@ManyToOne`	EAGER
`@OneToMany`	LAZY
`@ManyToMany`	LAZY

Say this:

> "I set `@ManyToOne` to LAZY explicitly, because the default is EAGER and that means every time I load a transaction I also load its customer, even when I only wanted the amount. On a list of a thousand transactions that is a thousand unnecessary joins or queries."

Real-time example. `Transaction` has a `@ManyToOne` to `Customer` and to `Account`. The recall list screen shows only reference number, amount and status — no customer data at all. With the EAGER default, every single row still dragged its customer and account across, which is a large part of why that page was slow. Setting them LAZY meant the list query loaded only what the screen actually displays.

The catch with LAZY: if you access a lazy field after the persistence context is closed, you get `LazyInitializationException`. The correct fixes are to fetch what you need in the query (a join fetch), or map to a DTO inside the transaction. The wrong fix is `spring.jpa.open-in-view=true`, which keeps the session open for the whole request and hides the problem while making it worse.

Q3. ●● The N+1 problem — your story

This is the most important answer in this file. Be able to explain it in 60 seconds.

What it is

```
List<Transaction> txns = txnRepository.findAll();       // 1 query - gets 100 transactions

for (Transaction t : txns) {
    System.out.println(t.getCustomer().getName());      // 1 query EACH - 100 more
}
```

One query to get the list, then N more queries — one per row — to load each row's relation. Hence **N+1**. With 100 transactions that is **101 database round trips** instead of one or two.

Why it is so slow

Each round trip has network latency. Even at 5ms each, 100 extra queries is half a second of pure waiting, and it grows linearly with the data. This is why a report that

was fine in testing crawls in production — the row count went up.

How to find it

> "Turn on `spring.jpa.show-sql` or set the Hibernate SQL logger to DEBUG, then load the page and count the queries. If you see the same query repeating with a different ID, that is N+1. I also looked at execution plans to confirm which queries were actually costly."

How to fix it — know all four

1. JOIN FETCH — the usual fix.

```
@Query("SELECT t FROM Transaction t JOIN FETCH t.customer WHERE t.status = :status")
List<Transaction> findByStatusWithCustomer(@Param("status") TxnStatus status);
```

One query, everything loaded.

2. `@EntityGraph` — the declarative version.

```
@EntityGraph(attributePaths = {"customer", "account"})
List<Transaction> findByStatus(TxnStatus status);
```

3. Batch fetching — turns N queries into N/batch queries.

```
spring.jpa.properties.hibernate.default_batch_fetch_size: 25
```

4. A DTO projection — the best option for a read-only report.

```
@Query("SELECT new com.bank.dto.TxnReportDto(t.id, t.amount, c.name) " +
       "FROM Transaction t JOIN t.customer c WHERE t.status = :status")
List<TxnReportDto> findReportData(@Param("status") TxnStatus status);
```

This selects only the columns you need, avoids loading full entities into the persistence context, and skips dirty-checking entirely.

⚠ **The trap with JOIN FETCH:** fetching **two** collections in one query produces a cartesian product. Fetch one collection at a time, or use batch fetching for the second. Mentioning this unprompted marks you out.

● Your version of the answer

> "On the recall reports, generation was taking about 15 seconds. I turned on SQL logging and saw the same query repeating once per row — a classic N+1, because the report walked each transaction's related entities. I replaced it with a projection query that joined and selected only the columns the report needed, added indexes on the columns in the WHERE and JOIN clauses, and rewrote a couple of inefficient JOINs. That took it from 15 seconds to 6. On the Business Card APIs the same pattern was costing 2 to 3 seconds per call, and fixing the N+1 plus setting the right associations to lazy brought it under a second."

Practise that until it is fluent. It answers "tell me about performance tuning", "tell me about Hibernate", and "tell me about a difficult problem" all at once.

Q4. ● `@Transactional`

```
@Transactional
public void recallTransaction(Long id) {
    Transaction txn = repo.findById(id).orElseThrow();
    txn.setStatus(RECALLED);                // no save() needed - dirty checking
    auditService.record(txn);
    // commit on normal return, rollback on a RuntimeException
}
```

The three things they ask:

1. What does it actually do? Spring creates a **proxy** around the bean. The proxy opens a transaction before the method, commits after it returns, and rolls back on failure.

2. When does it roll back? By default on **unchecked** exceptions (`RuntimeException` and `Error`) only. A checked exception **commits** unless you say `@Transactional(rollbackFor = Exception.class)`. This surprises people and is a favourite question.

3. ● Why does `@Transactional` not work when you call the method from inside the same class?

> "Because it works through a proxy. An external caller goes through the proxy, so the transaction starts. But an internal call — `this.otherMethod()` — bypasses the proxy entirely and calls the real method directly, so no transaction is started. The fix is to move that method into a separate bean, so the call goes through the proxy again."

That is a senior-level answer and interviewers remember it.

Propagation, briefly: `REQUIRED` is the default — join the existing transaction or start one. `REQUIRES_NEW` suspends the current one and starts an independent one, which is what you want for audit logging that must persist even if the main work rolls back.

Real-time example — the clearest use of `REQUIRES_NEW`. A recall attempt fails validation halfway through and the main transaction rolls back, which is correct: no partial state is written. But the **audit record of the attempt must survive**, because compliance needs to know someone tried. If the audit insert joins the main transaction it is rolled back with everything else. `REQUIRES_NEW` on the audit method gives it its own transaction that commits independently.

In a banking system that is not a nicety — an audit trail that disappears when something fails is worse than no audit trail, because you believe it.

Isolation levels, in one line each:
- `READ_UNCOMMITTED` — can see uncommitted data (dirty reads). Rarely used.
- `READ_COMMITTED` — only committed data. The usual default in PostgreSQL and Oracle.
- `REPEATABLE_READ` — the same row reads the same within a transaction. MySQL's default.
- `SERIALIZABLE` — full isolation, slowest.

Real-time example — why isolation matters here. Two approvers open the same card request and click Approve at the same moment. Both transactions read the status as PENDING, both see it as approvable, and both write APPROVED — so the request is approved twice and two approval records exist. That is a **lost update**.

The fixes are optimistic locking with an `@Version` column, so the second write fails and can be retried, or a `SELECT ... FOR UPDATE` pessimistic lock. Optimistic is usually the right choice, because genuine collisions are rare and it does not hold database locks.

Q5. Hibernate caching

- **First-level cache** — the persistence context. **Always on**, scoped to one transaction. Loading the same entity twice in one transaction hits the database once.
- **Second-level cache** — optional, shared across transactions, needs a provider like Ehcache or Hazelcast. Good for reference data that rarely changes, such as a list of branch codes. Not for data that changes often.
- **Query cache** — caches query results. Must be used with the second-level cache and is easy to get wrong.

Real-time example. Branch codes, currency codes and transaction-status descriptions are read on nearly every request and change perhaps twice a year. That is textbook second-level cache material. Transactions themselves are **never** cached that way — the data changes constantly and stale financial data is worse than a slow query.

Say this.

> "The first-level cache is the persistence context. It is always on and scoped to a single transaction, so loading the same entity twice in one transaction only hits the database once. That one is free and you get it whether you think about it or not.
>
> The second-level cache is optional and shared across transactions, and it needs a provider like Ehcache. It is worth it for reference data that is read constantly and changes rarely. I would not cache transactional data that way — stale financial data is worse than a slow query, and cache invalidation across instances is where the bugs live."

Q6. Spring Data JPA repositories

```java
public interface TxnRepository extends JpaRepository<Transaction, Long> {

    // Spring generates the query FROM THE METHOD NAME
    List<Transaction> findByStatusAndAmountGreaterThan(TxnStatus status, BigDecimal amt);

    Optional<Transaction> findByReferenceNumber(String ref);

    // your own JPQL when the name would get silly
    @Query("SELECT t FROM Transaction t WHERE t.createdAt BETWEEN :from AND :to")
    List<Transaction> findInRange(@Param("from") LocalDate from, @Param("to") LocalDate to);

    // native SQL when you need database-specific features
    @Query(value = "SELECT * FROM transactions WHERE MATCH(notes) AGAINST(:q)", nativeQuery = true)
    List<Transaction> search(@Param("q") String q);
}
```

Hierarchy: `Repository` → `CrudRepository` → `PagingAndSortingRepository` → `JpaRepository`. Each adds methods; `JpaRepository` adds JPA-specific ones like `flush()` and `saveAll()`.

Pagination, which you should mention for any list endpoint:

```java
Page<Transaction> page = repo.findByStatus(PENDING, PageRequest.of(0, 20,
Sort.by("createdAt").descending()));
```

Say this.

> "Spring Data generates the implementation from the method name, so `findByStatusAndAmountGreaterThan` needs no code at all. I use derived queries while the name stays readable, and switch to `@Query` with JPQL once the name would become unreadable — which is usually around three conditions. Native SQL only when I need something database-specific.
>
> The one thing I always add on a list endpoint is `Pageable`. Returning an unbounded collection is fine in testing and falls over in production when the table grows."

Q7. `save()` vs `saveAndFlush()`, and entity states

Three states: **transient** (a new object, not known to Hibernate), **persistent** (managed by the persistence context — changes are tracked automatically), **detached** (was persistent, but the context has closed).

Dirty checking is the thing to mention: for a managed entity you do not need to call `save()`. At flush time Hibernate compares the entity with its loaded snapshot and issues an UPDATE only for the fields that changed.

`saveAndFlush()` pushes the SQL to the database immediately instead of waiting for the transaction to commit. Useful when you need a generated ID within the same transaction.

Say this.

> "An entity is transient when it is a plain new object Hibernate does not know about, persistent once it is managed by the persistence context, and detached after the context closes. The important consequence is **dirty checking**: while an entity is managed I do not need to call `save()` at all. I change a field, and at flush time Hibernate compares it with the snapshot it loaded and issues an update for only the changed columns.
>
> `saveAndFlush` pushes the SQL immediately instead of waiting for the commit, which I use when I need a generated ID within the same transaction."

Q8. `getOne` / `getReferenceById` vs `findById`

`findById` hits the database and returns `Optional`. `getReferenceById` returns a **lazy proxy** without querying — useful when you only need to set a foreign key relationship and never read the object's fields. Touching a field on a proxy for a row that does not exist throws `EntityNotFoundException`.

Say this.

> " `findById` goes to the database and returns an `Optional`. `getReferenceById` returns a lazy proxy without querying at all, which is what I want when I only need to set a foreign key — for example attaching an existing customer to a new transaction. There is no point loading the whole customer row just to write its ID.
>
> The catch is that if the row does not exist you do not find out until you touch a field on the proxy, and then it throws `EntityNotFoundException` — possibly somewhere far from where you called it."

✓ Check yourself before moving on

1. Explain the N+1 problem, how you detect it, and **four** ways to fix it.
2. Tell your 15s → 6s story in under 60 seconds, with the mechanism.
3. Explain why `@Transactional` does nothing on an internal method call.
4. State the default rollback behaviour and how to change it.
5. Give the fetch-type defaults for all four relationship annotations.

05 · Microservices

Time needed: 60 minutes

Posting B lists microservices as a mandatory skill — *"(Java, Spring Boot and Microservices) OR (Java, Spring Boot)"* — so this is back to top priority. It is also all over your own resume: Eureka, API Gateway, JWT, and a three-service project you designed yourself. You will be asked about it whichever posting the interviewer is working from.

Both postings also say **"validate the architecture"**, which is a design-review conversation. Being able to state the **cost** of a design, not just its benefit, is what that question is really testing.

If you are short of time, the highest-value parts are **Q1** (the patterns and what you give up), **Q3** (circuit breaker), **Q4** (saga) and **Q7** (your Classifieds walkthrough).

First, the context: monolith vs microservices

The monolith

One deployable application containing everything: users, transactions, reporting, notifications. One codebase, one build, one database.

It is not a bad thing. It is simple to develop, simple to deploy, and a single transaction can span the whole system. Most applications should start here.

Where it hurts as it grows:
- One small change means redeploying everything.
- One memory leak in reporting takes down payments too.
- You cannot scale only the part that is busy — you scale the whole thing.
- A large team constantly blocks itself in one codebase.

Microservices

Split it into independent services, each owning one business capability, each with its own database, communicating over the network.

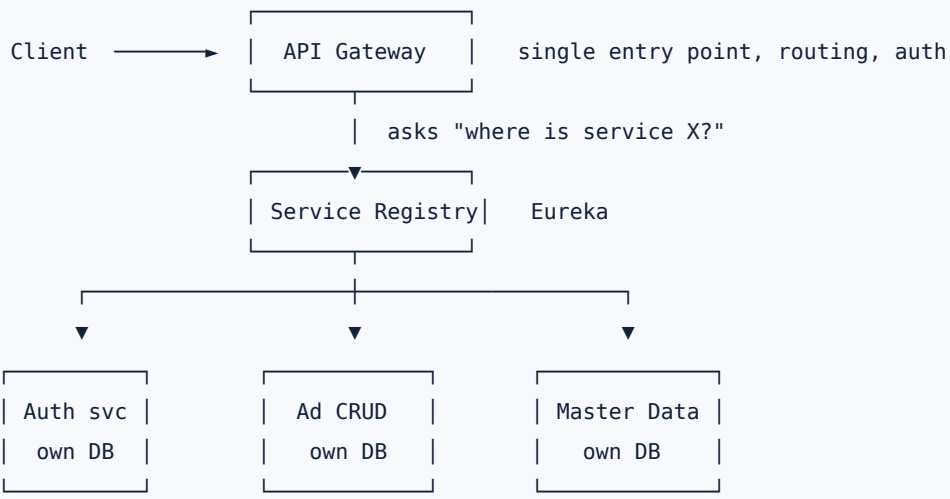
What you gain: independent deployment, independent scaling, fault isolation, and teams that can work without waiting for each other.

What you pay — say this part, because it shows judgement:

> "You are trading code complexity for **operational and distributed-systems** complexity. Network calls fail, so you need retries and circuit breakers. There is no single transaction across services, so you need eventual consistency. Debugging spans several services, so you need distributed tracing and centralised logging. It is the right trade at a certain scale, but for a small application a well-structured monolith is usually the better engineering decision."

That last sentence is what separates a candidate who has *used* microservices from one who has *read* about them. Interviewers at Infosys hear the buzzword version all day.

Q1. ● The core patterns — and which you used



1. Service Registry / Discovery — Eureka.
Services are on dynamic hosts and ports, and instances come and go. Hard-coding addresses does not work. Each service **registers** itself with Eureka on startup and sends heartbeats; callers **look up** the current instances by service name.

> "Without discovery I would be hard-coding hostnames, which breaks the moment a service scales or moves. With Eureka a caller asks for `ad-service` and gets a live list."

2. API Gateway.
One entry point for all clients. It handles **routing**, and centralises the things you do not want to repeat in every service: authentication, rate limiting, CORS, request logging.

> "Without a gateway the client would need to know every service's address, and every service would need its own copy of the auth logic. The gateway puts the cross-cutting concerns in one place."

(This is exactly the "Microservices API Management" the JD asks for. Say the words.)

3. Database per service.
Each service owns its data; no other service touches its tables. This is what makes independent deployment real — you can change your schema without coordinating with other teams.
The cost: no joins across services, and no single ACID transaction. Which leads to the next two patterns.

4. Config Server — externalised configuration for all services, so a change does not require a rebuild.

5. Circuit breaker — see Q3.

Q2. How do services talk to each other? ●

Synchronous — REST or gRPC. The caller waits for a reply.

```
@FeignClient(name = "master-data-service")     // declarative HTTP client
public interface MasterDataClient {
    @GetMapping("/categories/{id}")
    CategoryDto getCategory(@PathVariable Long id);
}
```

Simple, and you get an immediate answer. But it creates **temporal coupling**: if the callee is down, the caller fails too. Chains of synchronous calls multiply latency and failure probability.

Asynchronous — messaging over Kafka or RabbitMQ. The caller publishes an event and moves on.

> "The publisher does not know or care who consumes it, and if a consumer is down the message waits in the queue. That removes the temporal coupling. The cost is eventual consistency — the effect is not immediate — and harder debugging, because the flow is no longer a straight line."

How to choose — the rule to state:

> "If I need the answer to continue, synchronous. If I am telling the rest of the system that something happened, asynchronous. On the recall platform, publishing transaction state changes to Kafka for compliance logging and reporting was clearly the second kind — those consumers do not need to block the transaction from completing."

That is your project. Use it as the example.

Q3. ● Resilience — circuit breaker, retry, timeout

The problem: service A calls B, B becomes slow. A's threads pile up waiting. A runs out of threads and dies, so C which calls A dies too. One slow service takes down the system. That is a **cascading failure**.

Circuit breaker — three states, know all three:

State	Behaviour
Closed	Normal. Calls pass through, failures are counted.

State	Behaviour
Open	Failure threshold exceeded. Calls **fail immediately** without trying.
Half-open	After a wait, let a few test calls through. Success closes it, failure opens it again.

```
@CircuitBreaker(name = "masterData", fallbackMethod = "defaultCategory")
public CategoryDto getCategory(Long id) { return client.getCategory(id); }

public CategoryDto defaultCategory(Long id, Throwable t) {
    return CategoryDto.unknown();      // degrade gracefully instead of failing
}
```

Resilience4j is the current library; Hystrix is the older one, now retired.

> "Failing fast protects both sides. The caller does not exhaust its threads waiting, and the struggling service gets breathing room to recover instead of being hammered."

Real-time example. The advertisement service calls the master data service for category names. If master data goes down without a circuit breaker, every ad request hangs on that call until it times out, the ad service's thread pool fills, and the ad service stops serving requests it could have answered perfectly well. With a breaker and a fallback, ads still render — just with a placeholder category. **Degraded is much better than down**, and that sentence is the point of the whole pattern.

Also mention **timeouts** (never make a network call without one) and **retry with exponential backoff and jitter** — and that you only retry **idempotent** operations, because retrying a payment could charge someone twice.

Q4. ● Distributed transactions and the Saga pattern

The problem: placing an order must reserve stock, take payment and create a shipment — in three different services with three databases. There is no `@Transactional` that spans them.

Saga — break it into a sequence of local transactions, each publishing an event that triggers the next. If a step fails, run **compensating transactions** to undo the earlier ones.

```
Order created → Payment taken → Stock reserved → Shipment booked
                                       ✗ fails
              ← refund payment  ←  cancel order        (compensation)
```

Two flavours: **choreography** (services react to each other's events — simple, but the overall flow is not written down anywhere) and **orchestration** (a coordinator drives the steps — clearer and easier to debug, but it is another component to run).

Mention **eventual consistency**: the system is briefly inconsistent, and the business has to accept that. In banking that decision belongs to the business, not the developer.

Real-time example. A recall spanning services would be: mark the transaction as recalled, reverse the ledger entry, notify the customer. If the ledger reversal fails after the transaction was already marked, you cannot simply roll back across two databases — you publish a compensating event that returns the transaction to its previous state and records why. The compensation is itself an auditable business action, not a silent undo. In finance that is exactly how it has to work.

Idempotency matters here: a consumer may receive the same message twice, so processing must be safe to repeat. The usual approach is a unique message or request ID that you record and check before acting.

Say this.

> "Once each service has its own database there is no distributed transaction to fall back on, so a business operation spanning services becomes a sequence of local transactions, each publishing an event that triggers the next. If a step fails, you cannot roll the earlier ones back, so you run **compensating transactions** that undo them as new business actions.
>
> The two styles are choreography, where services react to each other's events, and orchestration, where a coordinator drives the sequence. Choreography is simpler to start but the overall flow is not written down anywhere, so I would prefer orchestration for anything with more than about three steps or anything auditable.
>
> The thing to be explicit about with the business is that this is eventual consistency — there is a window where the system is partly updated. That is a business decision, not a technical one, and in finance it needs to be an informed one."

Q5. Observability — the question people forget

> "Once a request crosses five services, a stack trace is not enough. You need three things. **Centralised logging**, so all logs land in one place — ELK or Splunk. **Distributed tracing** with a correlation ID passed through every call, so you can follow one request end to end — Sleuth with Zipkin, or OpenTelemetry. And **metrics and health checks**, through Actuator and Prometheus, so you see latency and error rates per service."

Your resume mentions resolving production incidents by analysing logs, so this connects.

Q6. Other things they may ask

How do you version an API? URL versioning (`/api/v1/...`) is the most common and the most visible. The real rule is to make **additive** changes — adding an optional field is backwards compatible; removing or renaming one is not.

How do you handle security between services? The gateway validates the JWT once, then services trust the network boundary, or pass the token along and each validates it.

Service-to-service can use mTLS.

What is the Strangler Fig pattern? How you migrate a monolith gradually: put a proxy in front, move one capability at a time to a new service, and route that path to the new one. The monolith shrinks until it disappears. Nobody sensible does a big-bang rewrite.

What is a bounded context? From domain-driven design — the boundary within which a model has one consistent meaning. It is the sane way to decide **where to split services**: by business capability, not by technical layer.

> "The wrong split is a 'database service' and a 'business logic service', because every change touches both. The right split is by business capability — orders, payments, notifications — so most changes stay inside one service."

How small should a service be? Not about lines of code:

> "Small enough that one team can own it and understand it fully, and large enough that a typical change does not require modifying three services at once. If every feature spans multiple services, the boundaries are wrong."

Q7. Your Classifieds project — how to present it

Have this ready as a 90-second walkthrough:

> "I built a three-service classifieds platform. An **authentication service** issuing and validating JWTs, an **advertisement service** with the CRUD operations, and a **master data service** for reference data like categories and locations. Each has its own database, so they can be deployed independently.
>
> **Eureka** handles service discovery, so services resolve each other by name rather than hard-coded addresses. An **API Gateway** sits in front as the single entry point and does the routing, which also means authentication is handled in one place rather than duplicated in each service.
>
> I added **global exception handling** with `@RestControllerAdvice` so all three services return the same error shape, and **DTO validation** with Bean Validation, so bad requests are rejected at the boundary rather than deep in the business logic. Security is a JWT layer with role-based access control across the services."

Be ready for the honest follow-ups:
- *"Did you deploy it?"* — answer truthfully. If it ran locally with Docker Compose, say that.
- *"How did you handle a service being down?"* — if you did not add a circuit breaker, say so and describe what you would add now. That is a better answer than inventing one.
- *"Why three services and not one?"* — it is a learning project; say so, and then show you know the trade-offs from the section above. Honesty plus understanding beats a bluff that collapses.

✓ Check yourself before moving on

1. Explain monolith vs microservices **including what you give up**.
2. Explain the three circuit breaker states and why failing fast helps both sides.
3. Explain the Saga pattern and compensating transactions.
4. Explain why you would use messaging instead of a REST call, with your Kafka example.
5. Give the 90-second Classifieds walkthrough without notes.

06 · Kafka and RabbitMQ

Time needed: 60 minutes

This is your **differentiator**. Most candidates at this level have Spring Boot and Hibernate. Far fewer have used both Kafka and RabbitMQ in production, and your resume says you used each for a *different reason*. That is the interesting part — be ready to explain the reasoning, not just name the tools.

First, the context: why messaging at all?

The problem you actually solved

An approval endpoint that sends an email synchronously:

```
@PostMapping("/approve/{id}")
public ResponseEntity<?> approve(@PathVariable Long id) {
    cardService.approve(id);          //  200 ms
    emailService.send(...);           //  2-4 seconds, and it can fail or hang
    return ResponseEntity.ok();       //  the user has been waiting the whole time
}
```

Three problems:
1. **The user waits** for something they do not care about.
2. **If the mail server is slow, the API times out** — and to the user it looks like the approval failed, even though it succeeded.
3. **Bulk approvals multiply it.** 50 approvals means 50 sequential email sends inside the request.

The fix

```
@PostMapping("/approve/{id}")
public ResponseEntity<?> approve(@PathVariable Long id) {
    cardService.approve(id);                        // 200 ms
    rabbitTemplate.convertAndSend("email.queue", new EmailMessage(...));  // ~1 ms
    return ResponseEntity.ok();                     // returns immediately
}
```

A separate consumer picks the message up and sends the email. The API responds in milliseconds. If the mail server is down, the message waits in the queue and is delivered when it recovers, instead of being lost.

Say it this way:

> "The email is not part of what the user is waiting for, so it should not be inside the request. Moving it to RabbitMQ meant the API returned immediately, the timeouts during bulk approvals stopped, and a mail server outage no longer failed the approval — the messages just queued up until it came back."

That is exactly what your resume says you did. Make sure the *reasoning* comes out, not just the tool name.

Q1. ●● Kafka vs RabbitMQ — when do you use which?

This is the question your resume invites. You used both, so you will be asked to justify it.

	RabbitMQ	Kafka
Model	Message **broker** / queue	Distributed **event log**
After consumption	Message is **removed**	Message **stays** for its retention period
Consumers	Usually one consumer per message	**Many** independent consumers read the same events
Replay	No	**Yes** — reset the offset and re-read
Routing	Rich — exchanges, routing keys, topics	Simple — topics and partitions
Ordering	Per queue	**Per partition**
Throughput	High	**Very high** — designed for streams
Best for	Task queues, work distribution, RPC-style	Event streaming, audit trails, multiple subscribers

The one-line distinction to memorise:

> "RabbitMQ is a **postbox** — a message is delivered to a worker and then it is gone. Kafka is a **ledger** — events are appended and stay there, and any number of consumers can read them at their own pace, including re-reading from the past."

● Your answer, using your own project

This is the strongest version, because it is true and it shows you chose deliberately:

> "I used them for two different jobs.
>
> **RabbitMQ for the email notifications.** That is a task queue: one message, one worker, do the job, and the message is done. I did not need to keep it or replay it — I needed the work off the request thread so bulk approvals stopped timing out.
>
> **Kafka for publishing transaction state changes.** That is an event stream, and several downstream consumers needed it — compliance logging and report generation, and potentially more later. With Kafka each consumer group reads independently at its own pace, and because the events are retained, a consumer that was down can catch up, and compliance can replay history if it needs to rebuild. With a queue, the first consumer would take the message and the others would never see it."

That last sentence is the technical heart of it. It shows you understand the difference rather than having used whatever the team had.

Q2. RabbitMQ concepts

```
Producer → Exchange → (routing rules) → Queue → Consumer
```

The **exchange** is the piece people forget. A producer never sends to a queue directly; it sends to an exchange, which decides which queues get a copy.

Exchange type	Routes by
Direct	Exact routing key match
Topic	Wildcard pattern — `txn.*.approved`
Fanout	Copies to **every** bound queue, ignoring the key
Headers	Header values

```java
@RabbitListener(queues = "email.queue")
public void handle(EmailMessage msg) {
    emailSender.send(msg);        // ack on normal return; requeue or DLQ on exception
}
```

Acknowledgements. The broker only removes a message once the consumer acknowledges it. If the consumer crashes mid-processing, the message is redelivered — so processing must be **idempotent**.

Dead Letter Queue (DLQ). Where messages go after repeated failures, instead of being retried forever or dropped silently. Mentioning DLQs unprompted signals production experience:

> "Without a DLQ a poison message either blocks the queue or gets retried indefinitely. With one, it moves aside after N attempts, gets alerted on, and someone can inspect it without holding up everything behind it."

Q3. Kafka concepts ●

```
Topic: transaction-events
├── Partition 0 → [ msg ][ msg ][ msg ]      ← ordered within the partition
├── Partition 1 → [ msg ][ msg ]
└── Partition 2 → [ msg ][ msg ][ msg ]
```

Topic — a named stream of events.
Partition — a topic is split into partitions, which is how Kafka scales. Order is guaranteed **within** a partition, not across the topic.
Offset — each consumer's position in a partition. Consumers commit their offsets, so they can resume where they left off.
Consumer group — consumers in the same group **share** the partitions, one partition per consumer at a time. Consumers in **different** groups each get **all** the messages.

That last point is the mechanism behind your answer in Q1: compliance and reporting are two different consumer groups, so both see every event.

● How do you guarantee ordering in Kafka?

> "By partition key. All messages with the same key go to the same partition, and order is guaranteed within a partition. So for transaction events I would key by account or transaction ID, which guarantees that everything about one transaction is processed in order, while different transactions still spread across partitions for throughput."

Delivery guarantees — know the three:
- **At most once** — commit the offset before processing. Fast, may lose messages.
- **At least once** — commit after processing. **The common default.** May duplicate, so consumers must be idempotent.
- **Exactly once** — possible with Kafka transactions, but costly and complex.

> "In practice most systems run at-least-once and make consumers idempotent, because exactly-once across a distributed system is expensive. For financial events I would record a processed-message ID and check it before acting, so a redelivery is a no-op."

Replication. Each partition has a leader and followers on other brokers. If the leader dies, a follower is promoted. That is Kafka's durability story.

Q3b. ActiveMQ and JMS — named in the JD

You have Kafka and RabbitMQ, which is the harder knowledge. ActiveMQ is a short add-on.

> "**ActiveMQ** is a traditional message broker and a **JMS** implementation — JMS being the Java standard API for messaging, so the code is portable across JMS brokers. Conceptually it is much closer to RabbitMQ than to Kafka: messages go to a **queue** (one consumer) or a **topic** (publish-subscribe), and once consumed and acknowledged they are gone.
>
> So the same distinction applies as with RabbitMQ — ActiveMQ is a broker that delivers and discards, Kafka is a retained log that many consumer groups read independently and can replay. I have used RabbitMQ and Kafka rather than ActiveMQ, but the model maps directly."

The JMS vocabulary, in case they use it: a `Queue` is point-to-point, a `Topic` is publish-subscribe, a **durable subscriber** receives messages published while it was offline, and acknowledgement modes control when a message is considered consumed.

Where ActiveMQ still shows up: older enterprise Java estates, especially in banking, where JMS was the standard long before Kafka existed. Saying that shows you understand *why* a BFSI client might still be on it.

Q4. Quick comparison questions

"Why not just use a database table as a queue?"

> "It works at small scale, and people do it. But you end up building polling, locking, retries, dead-lettering and back-pressure yourself, and polling a table is wasteful and adds latency. A broker gives you all of that, plus push delivery."

"What happens if a message fails?"
Retry with backoff, then a dead letter queue after N attempts, then alert. Never silently swallow.

"What if the same message arrives twice?"
Idempotency: a unique message ID recorded on the consumer side, or design the operation so applying it twice has the same effect as once.

"How do you not lose messages?"
Producer side: acknowledgements and retries. Broker side: persistence and replication. Consumer side: acknowledge only **after** successful processing.

✓ Check yourself before moving on

1. Kafka vs RabbitMQ in one sentence each, then **your** reason for using both.
2. Explain consumer groups and why two groups both receive every event.
3. Explain how ordering works in Kafka and how you would key transaction events.
4. Explain why at-least-once delivery means consumers must be idempotent.
5. Tell the email-timeout story with the reasoning, in 45 seconds.

07 · Spring Security and JWT

Time needed: 45 minutes

Your resume lists Spring Security with role-based access control, and JWT in your microservices project. Expect a few questions, especially in a banking context.

First, the context: authentication vs authorisation

- **Authentication** — *who are you?* Checking credentials.
- **Authorisation** — *what are you allowed to do?* Checking permissions.

You authenticate once; you authorise on every request. In your Business Card system, authentication proved the user was a bank employee; authorisation decided whether that employee was a **requester**, an **approver** or an **admin**.

Q1. How does Spring Security work?

It sits in front of your application as a **chain of servlet filters**. Every request passes through the chain before reaching a controller.

```
Request → [ filters: authenticate, authorise, CSRF, CORS ] → DispatcherServlet → Controller
```

The key filter for a token-based API is one that reads the token, validates it, and places an `Authentication` object into the `SecurityContext`. Everything downstream then knows who the caller is.

```java
@Configuration
@EnableWebSecurity
public class SecurityConfig {

    @Bean
    SecurityFilterChain chain(HttpSecurity http) throws Exception {
        http
          .csrf(csrf -> csrf.disable())                      // stateless API - see Q5
          .sessionManagement(s -> s.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
          .authorizeHttpRequests(auth -> auth
              .requestMatchers("/api/auth/**").permitAll()
              .requestMatchers("/api/cards/approve/**").hasRole("APPROVER")
              .requestMatchers("/api/admin/**").hasRole("ADMIN")
              .anyRequest().authenticated())
          .addFilterBefore(jwtFilter, UsernamePasswordAuthenticationFilter.class);
        return http.build();
    }
}
```

Say this.

> "Spring Security is a chain of servlet filters that runs before the request reaches a controller. For a token-based API the key filter reads the `Authorization` header, validates the token, and puts an `Authentication` object into the `SecurityContext` — after which everything downstream knows who the caller is and what roles they hold.
>
> The important point is that it runs **before** my code, so an unauthenticated request never reaches business logic at all. I do not have to remember to check in each method."

Q2. • Role-based access control — your story

Two ways to enforce it. Know both.

At the URL level, as above with `hasRole(...)`.

At the method level, which is more precise:

```java
@PreAuthorize("hasRole('APPROVER')")
public CardDto approve(Long id) { ... }

@PreAuthorize("hasRole('ADMIN') or #userId == authentication.principal.id")
public UserDto getUser(Long userId) { ... }     // admins, or the user themselves
```

Requires `@EnableMethodSecurity`.

Your version:

> "In the Business Card system there were three roles — requester, approver and admin — and the rule that mattered was that a requester must never be able to approve. I enforced it at the API layer with Spring Security rather than in the UI, because a UI check only hides the button. Anyone can call the endpoint directly with a REST client, so the authorisation has to be server side."

That last sentence is the point. In a banking interview it lands well.

⚠ **`hasRole` vs `hasAuthority`:** `hasRole("ADMIN")` looks for the authority `ROLE_ADMIN` — Spring adds the prefix for you. `hasAuthority("ROLE_ADMIN")` is the literal form. Mismatched prefixes are a very common bug.

Q3. ● JWT — how it works

A JWT is three base64 sections separated by dots:

```
header.payload.signature

header    { "alg": "HS256", "typ": "JWT" }
payload   { "sub": "neelima", "roles": ["APPROVER"], "exp": 1735689600 }
signature HMACSHA256(base64(header) + "." + base64(payload), secret)
```

The flow

```
1. POST /login with credentials
2. Server verifies, signs a JWT, returns it
3. Client sends it on every request:  Authorization: Bearer <token>
4. Server verifies the SIGNATURE and expiry - no database lookup needed
5. Request proceeds with the user's identity and roles
```

● Why use a JWT instead of a session?

> "A session means the server stores state and every request looks it up, which is fine for one server but awkward when you scale — you need sticky sessions or a shared store like Redis. A JWT is **self-contained**: the user and roles are inside it, and the signature proves it has not been tampered with. So any instance of any service can validate it independently, with no shared session store. In a microservices setup that is the deciding factor."

● The two things people get wrong

1. A JWT is signed, not encrypted. Anyone can base64-decode the payload and read it. The signature only proves it has not been *altered*. **Never put anything sensitive in the payload** — no passwords, no account numbers.

2. You cannot easily revoke one. It is valid until it expires, so if a token is stolen it works until then. The usual answers: keep access tokens **short-lived** (15 minutes) with a longer-lived **refresh token**, and keep a small **blocklist** of revoked token IDs for logout. Say the trade-off rather than pretending it is solved.

Q4. Where should the token be stored on the client?

> "An `httpOnly` cookie is safer than `localStorage`, because JavaScript cannot read an httpOnly cookie, so an injected script cannot steal it. Anything in localStorage is readable by any script on the page. The trade-off is that cookies need CSRF protection, whereas an `Authorization` header does not."

Q5. CSRF, and why it is disabled for APIs

CSRF is when another site makes the user's browser send a request to yours, relying on the browser automatically attaching the session cookie.

> "It is disabled for a stateless API because the attack depends on the browser attaching credentials automatically. If the client sends an explicit `Authorization` header, another site cannot make the browser add it. If I were using cookie-based auth, CSRF protection would need to stay on."

Do not just say "we disable CSRF" — say *why*, or it looks like copied configuration.

Q6. Password storage

```
@Bean PasswordEncoder passwordEncoder() { return new BCryptPasswordEncoder(); }
```

Never store plain text, and never a plain hash like MD5 or SHA-256. BCrypt is deliberately **slow** and **salts** each password automatically, which defeats rainbow tables and makes brute force expensive.

Say this.

> "Passwords are stored as a BCrypt hash, never plain text and never a plain SHA or MD5 hash. Two properties matter. BCrypt salts each password automatically, so two users with the same password get different hashes and a precomputed rainbow table is useless. And it is deliberately **slow** with a tunable work factor, so brute forcing is expensive — which is the opposite of what you want from a general-purpose hash. That slowness is the feature."

Q6b. ●● OAuth2 — named in the JD, so know the difference from JWT

The confusion to clear up first, because it is the question:

> "They are not alternatives. **OAuth2 is a framework for how a token is obtained**, and **JWT is a format a token can be written in**. You can do OAuth2 with opaque tokens, and you can use JWTs without OAuth2 — which is what I did, issuing them directly after a login."

What OAuth2 is actually for

> "The problem it solves is **delegated access** — letting an application act on a user's behalf without the user handing over their password. Instead of giving the app your credentials, you authorise it with the identity provider, and the app gets a token that is limited in scope and expires."

The four roles — name them

Role	Who
Resource owner	The user
Client	The application asking for access
Authorization server	Issues the token — Keycloak, Okta, Azure AD
Resource server	Your API, which validates the token

The flow to know: Authorization Code

```
1. App redirects the user to the authorization server
2. User logs in there  — the app NEVER sees the password
3. Authorization server redirects back with a short-lived CODE
4. App exchanges the code (plus its client secret) for an ACCESS TOKEN
5. App calls your API with  Authorization: Bearer <token>
6. Your API validates it and serves the request
```

Why the extra code step? So the token is never exposed in a browser URL or in history — the exchange happens server to server. For mobile and single-page apps, which cannot keep a client secret, the same flow is used **with PKCE**, which proves the exchanger is the same app that started the flow.

Other grant types, one line each: *client credentials* for service-to-service with no user involved — worth naming, because that is how microservices authenticate to each other. *Refresh token* to get a new access token without the user logging in again.

Access token vs refresh token vs ID token

- **Access token** — short lived (minutes), sent on every API call, proves what you may do.
- **Refresh token** — long lived, kept safely, used only to get a new access token.
- **ID token** — an OpenID Connect concept, not OAuth2 itself: it says *who* the user is.
  OIDC is an identity layer on top of OAuth2 — OAuth2 is authorisation, OIDC is

authentication. That distinction is a strong thing to know.

In Spring

```
// your API as a resource server, validating JWTs issued by an authorization server
http.oauth2ResourceServer(oauth2 -> oauth2.jwt(Customizer.withDefaults()));
```

```
spring.security.oauth2.resourceserver.jwt.issuer-uri: https://auth.example.com/realms/bank
```

Spring fetches the issuer's public keys (the **JWKS** endpoint) and validates the signature, issuer and expiry on every request. No call to the auth server per request, and no shared secret — which is why it scales across microservices.

● Your honest answer

> "In my work I implemented authentication and role-based access control with Spring Security and JWTs issued by our own service, rather than integrating a full OAuth2 authorization server. So I understand the model and the flows, but I have not stood up Keycloak or integrated with Azure AD. In a BFSI context I would expect a central identity provider, with my service acting as a resource server validating tokens against its public keys."

That answer is honest, it uses the right vocabulary, and it ends on what you would do.

Q7. Other things worth a sentence

- **OAuth2 vs JWT** — not alternatives. OAuth2 is the authorisation *framework*; a JWT is a token *format* often used within it.
- **CORS vs CSRF** — CORS is a browser rule about which origins may read a response. CSRF is an attack. They get confused constantly.
- **Method security vs URL security** — URL rules are coarse and can be bypassed if another entry point reaches the same service method. Method-level annotations protect the method itself.

Say this if they probe the distinctions.

> "OAuth2 and JWT are not alternatives — OAuth2 is an authorisation framework describing how a token is obtained, and JWT is a token format often used inside it. CORS and CSRF also get confused: CORS is a browser rule about which origins may read a response, and CSRF is an attack where another site makes the browser send an authenticated request. And URL-level security is coarse — if another entry point reaches the same service method it is unprotected, so for anything sensitive I put `@PreAuthorize` on the method itself."

✓ Check yourself before moving on

1. Explain the JWT structure and the full request flow.
2. Explain why a JWT suits microservices better than server-side sessions.
3. Explain why a JWT is not secret, and what that means for what you put in it.
4. Explain why CSRF is disabled for a stateless API.
5. Tell your RBAC story, ending on why authorisation must be server side.

08 · SQL and Query Optimisation

Time needed: 75 minutes

Your resume claims SQL tuning, indexing and execution plan analysis. That is unusual at this level and it is genuinely valuable — but it also means you **will** be tested on it. An interviewer who sees "identified and removed N+1 queries, added strategic indexing" will ask you to explain indexing properly.

Q1. Joins — know all four

```sql
-- INNER: only rows that match in both tables
SELECT t.id, c.name FROM transactions t
INNER JOIN customers c ON t.customer_id = c.id;

-- LEFT: every row from the left, NULLs where the right has no match
SELECT c.name, t.id FROM customers c
LEFT JOIN transactions t ON t.customer_id = c.id;     -- includes customers with no txns

-- RIGHT: the mirror image. Rarely used - people rewrite it as a LEFT JOIN.
-- FULL OUTER: everything from both sides. MySQL does not support it directly.
```

The classic question: *"Find customers with no transactions."*

```sql
SELECT c.* FROM customers c
LEFT JOIN transactions t ON t.customer_id = c.id
WHERE t.id IS NULL;
```

The `WHERE ... IS NULL` after a LEFT JOIN is the pattern. Learn it.

⚠ **A trap worth knowing:** putting a condition on the right-hand table in the `WHERE` clause of a LEFT JOIN silently turns it into an INNER JOIN, because `NULL` fails the condition. If you want to filter the joined table without losing left rows, the condition belongs in the `ON` clause.

Say this.

> "An inner join returns only rows that match on both sides. A left join returns every row from the left table and nulls where the right has no match, which is what I use when the absence is the thing I care about — customers with no transactions, or requests with no approval yet. Right joins are the mirror image and people usually rewrite them as left joins for readability.
>
> The trap is putting a condition on the right-hand table in the `WHERE` clause of a left join. Null fails the condition, so those rows are dropped and you have silently turned it back into an inner join. The condition has to go in the `ON` clause."

Q2. ●● Indexing — the answer that matters most here

What an index actually is

> "An index is a separate sorted structure, usually a **B-tree**, that maps column values to row locations. Without one, the database does a **full table scan** — it reads every row to find matches, which is O(n). With one, it walks the tree, which is O(log n). It is the same reason you use the index at the back of a book instead of reading every page."

When an index helps

- Columns in a `WHERE` clause
- Columns you `JOIN` on — **especially foreign keys, which are not indexed automatically in MySQL's InnoDB unless you add them**
- Columns in `ORDER BY` and `GROUP BY`

● The cost — always mention this, it is what shows real understanding

> "Indexes are not free. Every `INSERT`, `UPDATE` and `DELETE` has to update every index on that table, so over-indexing slows down writes. They also take disk space. On a write-heavy table I index deliberately, not by default — the right number is the smallest set that serves the actual query patterns."

Saying only "indexes make queries faster" is the answer of someone who read it. Saying "and they slow down writes, so I add them deliberately" is the answer of someone who has tuned a real system.

Composite indexes and the left-most prefix rule

```
CREATE INDEX idx_txn_status_date ON transactions (status, created_at);
```

This index serves:
- `WHERE status = 'PENDING'` ✓
- `WHERE status = 'PENDING' AND created_at > '2026-01-01'` ✓

But **not** `WHERE created_at > '2026-01-01'` alone ❌ — because the index is sorted by `status` first. This is the **left-most prefix rule**, and it is a favourite question. Column order in a composite index matters.

Real-time example. The recall report always filters by status and then by a date range: `WHERE status = 'PENDING' AND created_at BETWEEN ? AND ?`. So the index is `(status, created_at)` in that order — the equality column first, the range column second. Reversed, as `(created_at, status)`, the index is far less effective, because the range scan happens first and status cannot narrow it. Getting that column order right was part of the 15-second to 6-second fix.

What stops an index being used

```
WHERE YEAR(created_at) = 2026        -- ❌ a function on the column defeats the index
WHERE created_at >= '2026-01-01'     -- <span class="tick">✓</span> same meaning, index usable

WHERE account_number LIKE '%1234'    -- ❌ a leading wildcard cannot use a B-tree
WHERE account_number LIKE '1234%'    -- <span class="tick">✓</span> prefix match is fine
```

Mentioning these two unprompted is a strong signal.

Clustered vs non-clustered

- **Clustered** — the table rows are physically stored in this order. One per table; in InnoDB it is the primary key.
- **Non-clustered** — a separate structure pointing back to the row.

Covering index

An index containing every column a query needs, so the database answers it from the index alone and never touches the table. Sometimes the biggest single win available.

Real-time example. A dashboard count of pending recalls per branch needs only `status` and `branch_code`. An index on `(status, branch_code)` covers it, so the database answers from the index and never reads a single table row. On a large transactions table that is the difference between milliseconds and seconds.

Q3. ● EXPLAIN — how you actually diagnose

Your resume says you analysed execution plans. Be ready to describe the process.

```
EXPLAIN SELECT t.id, c.name FROM transactions t
JOIN customers c ON t.customer_id = c.id
WHERE t.status = 'PENDING';
```

What you look for:

Column	What matters
`type`	`ALL` = full table scan, the thing to fix. `ref` or `range` is good. `const` / `eq_ref` is best.
`key`	Which index was actually used. `NULL` means none.
`rows`	Estimated rows examined. A big number here is the problem.
`Extra`	`Using filesort` or `Using temporary` mean extra work. `Using index` is a covering index — good.

Say the process, not just the tool:

> "I start from the slow query log or the application timings to find *which* query is slow — there is no point optimising the wrong one. Then I run `EXPLAIN` on it. If the type is `ALL` and the row count is large, it is doing a full scan, and I look at what the `WHERE` and `JOIN` columns are and whether an index exists. After adding one I re-run `EXPLAIN` to confirm it is actually being used, and re-measure the query time. Adding an index without checking the plan afterwards is guessing."

Q4. Normalisation, and when to break it

- **1NF** — atomic values, no repeating groups.
- **2NF** — 1NF, and no partial dependency on part of a composite key.
- **3NF** — 2NF, and no non-key column depending on another non-key column.

> "Normalisation removes duplication, so there is one place to update a fact and no inconsistency. The cost is more joins. **Denormalisation** deliberately duplicates data to avoid expensive joins on read-heavy paths — for example storing a customer name on a report table so a reporting query does not join. It is a trade: faster reads, but now you have two copies to keep in sync."

Q5. Common query questions

Second-highest salary — asked constantly:

```sql
SELECT MAX(salary) FROM employees WHERE salary < (SELECT MAX(salary) FROM employees);

-- or, more generally, the Nth highest
SELECT DISTINCT salary FROM employees ORDER BY salary DESC LIMIT 1 OFFSET 1;
```

Find duplicates:

```sql
SELECT email, COUNT(*) FROM customers GROUP BY email HAVING COUNT(*) > 1;
```

- **`WHERE` vs `HAVING`:** `WHERE` filters **rows before** grouping. `HAVING` filters **groups after** aggregation. You cannot use an aggregate in `WHERE`.

`UNION` vs `UNION ALL`: `UNION` removes duplicates, which costs a sort. `UNION ALL` keeps everything and is faster. Use `UNION ALL` unless you actually need deduplication.

`DELETE` vs `TRUNCATE` vs `DROP`: `DELETE` removes rows, can have a `WHERE`, is logged and can be rolled back. `TRUNCATE` empties the table quickly, no `WHERE`, resets auto-increment. `DROP` removes the table itself.

Window functions, worth knowing one:

```sql
SELECT name, salary, department,
       RANK() OVER (PARTITION BY department ORDER BY salary DESC) AS rank_in_dept
FROM employees;
```

Unlike `GROUP BY`, a window function keeps every row and adds a computed column.

Say this when they give you a query to write.

> "Let me state what I am aiming for first — I want the second highest distinct salary, so I need to handle duplicates, and decide what should happen if there is no second value."

Then write it. The pattern that scores is: restate, mention the edge case, write, then say the complexity or the index it would need. For the duplicates query, mention that `GROUP BY ... HAVING COUNT(*) > 1` is the standard shape and that on a large table the grouped column wants an index.

Q6. ACID

- **Atomicity** — all of the transaction, or none of it.
- **Consistency** — the database moves from one valid state to another; constraints hold.
- **Isolation** — concurrent transactions do not corrupt each other.
- **Durability** — once committed, it survives a crash.

The banking example makes this concrete: a transfer is a debit and a credit. Without atomicity, a crash between them destroys money. Use that example — it fits your domain.

Say this — use the banking example, it is your domain.

> "A transfer is the clearest example. Debiting one account and crediting another must be **atomic** — both or neither, because a crash between them destroys money. **Consistency** means constraints still hold afterwards, so the ledger still balances. **Isolation** means two concurrent transfers on the same account cannot interleave and produce a wrong balance. And **durability** means once it is committed it survives a crash — the customer has been told it happened, so it has to have happened.
>
> That is also why I would not move something like a ledger entry to eventual consistency without the business explicitly agreeing to it."

Q6b. ● The JD names MySQL, PostgreSQL and Oracle — you have MySQL

SQL skill transfers almost entirely. Say so, and know the dialect differences, because that is what proves you understand *what* varies rather than bluffing.

> "My production work has been MySQL, with MongoDB alongside it. I have not used Oracle or PostgreSQL specifically, but the SQL itself is largely the same — joins, indexing, execution plans and query tuning all transfer directly. The differences I would expect are dialect-level."

	MySQL	PostgreSQL	Oracle
Auto id	`AUTO_INCREMENT`	`SERIAL` / `IDENTITY`	**Sequence** + identity
Paging	`LIMIT 10 OFFSET 20`	`LIMIT / OFFSET`	`OFFSET ... FETCH FIRST`, or `ROWNUM`
Null fallback	`IFNULL`	`COALESCE`	`NVL`
A row from nothing	`SELECT 1`	`SELECT 1`	`SELECT 1 FROM DUAL`
Plan	`EXPLAIN`	`EXPLAIN ANALYZE`	`EXPLAIN PLAN FOR` + `DBMS_XPLAN`
String join	`CONCAT`	`||`	`||`

Two deeper differences worth knowing:
- **PostgreSQL** has strong support for JSON columns and proper window functions, and its default isolation is READ COMMITTED with MVCC — readers never block writers.
- **Oracle** uses `VARCHAR2`, treats the empty string as NULL (which surprises people), and PL/SQL stored procedures are far more common in enterprise Oracle estates — in a BFSI client you may well meet business logic living in the database.

Hibernate softens most of this, which is the practical point:

> "With JPA and Hibernate, the dialect is configuration — Hibernate generates the right SQL for the database. So moving between them mostly affects native queries and migration scripts rather than the application code. Where it bites is anything database-specific I wrote by hand."

Q7. SQL vs NoSQL — you have MongoDB on your resume

> "I would use a relational database when the data is structured and relational and I need ACID transactions — which is most of banking. MongoDB suits flexible or evolving schemas, document-shaped data, and cases where horizontal scaling matters more than joins. The real question is the access pattern: if I am constantly joining, that is a relational shape, and forcing it into documents means duplicating data everywhere."

Q8. Pagination at the database level

```sql
SELECT * FROM transactions ORDER BY created_at DESC LIMIT 20 OFFSET 0;
```

> "OFFSET pagination degrades on deep pages, because the database still has to scan and discard everything before the offset. Page 5,000 is slow. **Keyset pagination** — `WHERE created_at < :lastSeen ORDER BY created_at DESC LIMIT 20` — stays fast because the index seeks straight to the position."

That is a strong detail for a reporting-heavy system, which yours was.

✓ Check yourself before moving on

1. Explain what an index is, **and its cost on writes**.
2. Explain the left-most prefix rule with the composite index example.
3. Give two things that stop an index being used.
4. Describe your EXPLAIN process end to end.
5. Write the "customers with no transactions" query from memory.
6. Explain `WHERE` vs `HAVING`.

09 · Docker, Kubernetes and Containerisation

Time needed: 90 minutes

This is your biggest gap against this job description, and it is named three times — *"Docker, Kubernetes, Containerization"* in the required skills, *"familiarity with containerization technologies"* in the ideal-candidate list, and *"support DevOps practices"* in the responsibilities.

Your resume does not mention any of it. So the goal of this file is not to pretend — it is to understand the model well enough to hold a sensible conversation, and to say honestly where your hands-on ends. At 2 years, nobody expects you to run a cluster. They expect you to know what a container is and why your Spring Boot service is put in one.

> 🔴 **The answer that protects you all day:**
> *"I have not owned the container or deployment side — our platform was deployed by a separate team. So I would call that understanding rather than hands-on. What I do know is the model and why it is used, and I have built the applications that go inside the containers."*
>
> Say that **before** they probe, not after. Naming the boundary first makes everything else you say credible.

Part A · Docker

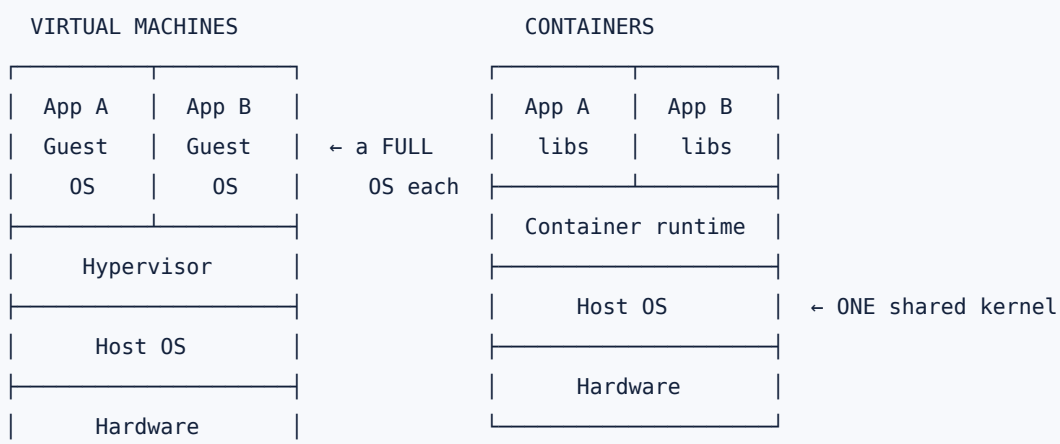
First, the context: what problem does a container solve?

The oldest complaint in software: **"it works on my machine."**

Your application needs a particular Java version, particular environment variables, a particular OS library. The server has slightly different ones. It works in SIT and fails in UAT, and you lose a day finding out why.

A **container** packages the application **together with everything it needs to run** — the JRE, the libraries, the configuration — into one image. That image runs identically wherever there is a container runtime. The machine underneath stops mattering.

Container vs virtual machine — a near-certain question



> "A virtual machine includes a whole guest operating system, so it is heavy — gigabytes, and a minute or more to boot. A container shares the host's kernel and packages only the application and its dependencies, so it is tens or hundreds of megabytes and starts in seconds. The trade-off is isolation: VMs isolate more strongly because they do not share a kernel. For running many instances of the same service, containers win on density and start-up time, which is what makes auto-scaling practical."

The three words you must not mix up

Term	What it is	The analogy
Dockerfile	The recipe — text instructions to build the image	The recipe
Image	The built, read-only package	The cake you baked
Container	A running instance of an image	A slice being eaten

One image, many containers. That is the whole point — the same image runs in SIT, UAT and production, and you scale by running more containers from it.

A Dockerfile for a Spring Boot app — know roughly what it looks like

```
# multi-stage build: stage 1 compiles, stage 2 only carries the result
FROM maven:3.9-eclipse-temurin-17 AS build
WORKDIR /app
COPY pom.xml .
RUN mvn dependency:go-offline        # cached layer - deps rarely change
COPY src ./src
RUN mvn clean package -DskipTests

FROM eclipse-temurin:17-jre-alpine   # small runtime image, no Maven, no JDK
WORKDIR /app
COPY --from=build /app/target/*.jar app.jar
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.jar"]
```

Two things here are worth understanding, because they are what an interviewer asks about:

1. Why a multi-stage build? The first stage needs Maven and the full JDK to compile — about 700MB of tooling. The final image needs none of that, only the JRE and the JAR. Multi-stage means the build tools are thrown away and the shipped image is a fraction of the size. Smaller image means faster pulls, faster deploys, and a smaller attack surface.

2. Why copy `pom.xml` and download dependencies before copying `src`? Because **Docker caches each instruction as a layer**. Dependencies change rarely; source changes constantly. If you copy everything at once, every code change re-downloads every dependency. Split like this, the dependency layer is reused and the rebuild is seconds instead of minutes.

That layer-caching answer is the single best thing you can say about Docker, because it shows you understand how images are actually built rather than having copied a file.

The commands to recognise

```
docker build -t recall-service:1.2 .     # build an image from the Dockerfile here
docker run -p 8080:8080 recall-service   # run it, mapping host port -> container port
docker ps                                # what is running
docker logs -f <container>               # follow the logs - your tail -f equivalent
docker exec -it <container> sh           # get a shell inside a running container
docker images                            # what images exist locally
```

`-p 8080:8080` is **host port : container port**. The container has its own network namespace, so without the mapping nothing on the host can reach it.

Configuration — the point that connects to Spring Boot

> "The image must be the same in every environment, so the environment-specific parts cannot be baked into it. Database URLs, credentials and feature flags come in as **environment variables** at run time. That maps directly onto Spring Boot profiles and `application.yml` — the same artefact is promoted from SIT to UAT to production and only the configuration changes. That is the real discipline containers enforce."

And never put secrets in an image. Anyone who can pull it can read every layer.

Part B · Kubernetes

First, the context: what problem does Kubernetes solve?

Docker runs **one container on one machine**. Now run 40 services, three replicas each, across a dozen machines. Something has to decide which machine each container runs on, restart the ones that die, replace them during a release without downtime, and give them stable addresses when their IPs keep changing.

That is orchestration, and Kubernetes is the orchestrator. The word to use is *declarative*: you describe the desired state — "I want three replicas of this image" — and Kubernetes continuously works to make reality match. If a pod dies, it starts another, because reality stopped matching the declaration.

The objects you must be able to name

Object	What it is
Pod	The smallest unit — usually one container. **Pods are disposable** and get a new IP each time
ReplicaSet	Keeps N copies of a pod running
Deployment	Manages ReplicaSets; this is what gives you rolling updates and rollback
Service	A **stable** name and IP in front of changing pods, with load balancing
Ingress	Routes outside HTTP traffic to Services — host and path based
ConfigMap	Non-secret configuration injected as env vars or files
Secret	The same, for sensitive values (base64 encoded, not encrypted by default)
Namespace	A logical partition of the cluster — dev, SIT, UAT

● The question they will actually ask: "Why do you need a Service?"

> "Because pods are disposable. Every time a pod is replaced it gets a new IP, so nothing can depend on a pod's address. A Service gives a stable DNS name and IP, and load balances across whichever pods are currently healthy. So one service calls another by name — `http://recall-service`
> — and Kubernetes resolves it, exactly the role Eureka played in my Classifieds project."

That connection is strong — it ties a named requirement to something you actually built.

A Deployment, roughly

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: recall-service
spec:
  replicas: 3
  template:
    spec:
      containers:
        - name: recall-service
          image: registry/recall-service:1.2
          ports: [{ containerPort: 8080 }]
          env:
            - name: SPRING_PROFILES_ACTIVE
              value: uat
          readinessProbe:                  # is it ready for traffic yet?
            httpGet: { path: /actuator/health/readiness, port: 8080 }
          livenessProbe:                   # is it still alive, or should it be restarted?
            httpGet: { path: /actuator/health/liveness, port: 8080 }
          resources:
            requests: { memory: "512Mi", cpu: "250m" }
            limits:   { memory: "1Gi",   cpu: "500m" }
```

●● Probes — the best Kubernetes answer available to a Spring Boot developer

This is where your actual knowledge connects, because **Spring Boot Actuator provides these endpoints**.

> "**Readiness** asks 'can this pod take traffic yet?' A Spring Boot service takes time to start — context initialisation, the connection pool, Liquibase migrations. Until readiness passes, Kubernetes keeps it out of the Service's load balancing, so no request hits a service that is not ready.
>
> **Liveness** asks 'is this still alive?' If it fails repeatedly, Kubernetes restarts the pod.
>
> Getting them the wrong way round is a classic production incident: if a slow startup fails a *liveness* probe, Kubernetes keeps killing and restarting the pod and it never comes up."

That last sentence is a real, specific failure mode, and it will land well.

Rolling updates and rollback

> "A Deployment replaces pods gradually rather than all at once — it brings up new ones, waits for readiness, then takes old ones down. So there is no downtime, and if the new version fails readiness the rollout stops rather than taking the service down. `kubectl rollout undo` goes back to the previous version.
>
> The thing that has to be true for this to work is that **both versions run at the same time for a period**. So a database migration has to be backwards compatible — add a nullable column, deploy, then use it. That is the same expand-then-contract discipline we used with Liquibase."

That is an excellent answer — it takes a Kubernetes question and answers it with something you genuinely did.

Scaling

`kubectl scale deployment recall-service --replicas=5`, or a **HorizontalPodAutoscaler** that adds pods when CPU or memory passes a threshold.
The prerequisite to mention: the service has to be **stateless** for this to work. Any session state must live outside the pod — in a database or a cache — because the next request may land on a different pod. That is the same reason Spring beans should be stateless, and the same reason a JWT suits this architecture better than a server-side session.

The commands to recognise

```
kubectl get pods                      # what is running
kubectl describe pod <name>           # why is it not running - events are at the bottom
kubectl logs -f <pod>                 # follow the logs
kubectl logs <pod> --previous         # logs of the crashed instance - this is the one for
CrashLoopBackOff
kubectl exec -it <pod> -- sh          # shell inside
kubectl rollout undo deployment/<name>
```

Two statuses to recognise, because they come up as scenario questions:
- **CrashLoopBackOff** — the container keeps starting and dying. Check `logs --previous`;

usually a missing environment variable, a database it cannot reach, or a failed migration.
- **ImagePullBackOff** — it cannot fetch the image. Wrong tag, or registry credentials missing.

✓ Check yourself before moving on

1. Container vs VM, including the isolation trade-off.
2. Dockerfile vs image vs container, and **why layer caching means you copy `pom.xml` first**.
3. Why a Service exists — and the Eureka comparison.
4. Readiness vs liveness, and the failure that happens when you confuse them.
5. Why rolling updates force backwards-compatible database migrations.
6. **Your honest boundary statement**, said before they have to ask for it.

10 · CI/CD and DevOps

Time needed: 45 minutes

The JD names **Jenkins, GitHub Actions and Azure DevOps**, and lists *"Implement CI/CD pipelines and support DevOps practices"* as a responsibility.

Your resume does not mention any pipeline tool. But you have been through SIT and UAT release cycles and done deployment validation, so you have lived inside a release process even if you did not own the pipeline. **That is the honest and sufficient answer at your level.**

> ● **The boundary statement:**
> *"I worked inside the release process rather than owning the pipeline — our builds and deployments went through SIT and UAT cycles, and I did the deployment validation after a release. So I understand the shape of it and I am comfortable working in one, but I have not built a pipeline from scratch."*

First, the context: what CI and CD actually mean

Continuous Integration (CI) — every commit is automatically built and tested. The point is to find out within minutes that something broke, instead of at the end of a sprint when ten people's changes are tangled together.

Continuous Delivery (CD) — every build that passes is automatically turned into a deployable artefact and pushed to an environment. Delivery means it is *ready* to release; **Continuous Deployment** means it goes to production automatically with no human gate.

In BFSI, the last step almost never happens automatically — production releases are gated and approved, because the environment is regulated. **Say that if asked; it shows you understand the context rather than reciting the textbook.**

The pipeline shape — this is what you need to know, whatever the tool

```
commit
  |
  ▼
BUILD        mvn clean package      compile, run unit tests
  |
  ▼
TEST         integration tests, code coverage
  |
  ▼
QUALITY      static analysis, security scan, coverage gate
  |
  ▼
PACKAGE      build the Docker image, tag it, push to a registry
  |
  ▼
DEPLOY       SIT  →  UAT  →  (approval)  →  PROD
```

The principle that matters most

> "**The same artefact is promoted through the environments.** You build once, and that exact image moves from SIT to UAT to production — only the configuration changes, through environment variables. If each environment rebuilt from source you would be testing one thing and shipping another, and 'it passed in UAT' would mean nothing."

That is the single best sentence you can say about CI/CD, and it ties straight back to the container discipline in file 09.

The three tools, in one line each

Tool	What it is
Jenkins	The long-established one. Self-hosted, plugin-driven, pipelines written as a `Jenkinsfile` in Groovy. Very flexible, and you maintain the server yourself.
GitHub Actions	Built into GitHub. YAML workflows in `.github/workflows/`, triggered by events — push, pull request, schedule. Hosted runners, no server to maintain.
Azure DevOps	Microsoft's suite — Repos, Pipelines, Boards, Artifacts. YAML pipelines. Common in enterprises already on Azure.

They do the same job. If asked which you prefer, the honest answer is better than a preference you cannot defend:

> "I have worked within a pipeline rather than choosing the tool. They all express the same shape — build, test, quality gate, package, deploy. Jenkins is the most flexible and the most to maintain; GitHub Actions is the simplest if the code is already on GitHub; Azure DevOps makes sense in a Microsoft shop. I would be comfortable picking up any of them."

Know roughly what a pipeline file looks like

```yaml
# GitHub Actions
name: build
on: [push, pull_request]

jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-java@v4
        with: { java-version: '17', distribution: 'temurin', cache: 'maven' }
      - run: mvn clean verify                    # compile + unit + integration tests
      - run: docker build -t registry/recall:${{ github.sha }} .
```

Two details worth noticing, because they are what someone asks about:
- `cache: 'maven'` — the dependency cache, for the same reason as Docker layer caching: without it every build re-downloads the world.
- **Tagging the image with the commit SHA** — so every deployed artefact traces back to exactly one commit. In a regulated environment that traceability is not optional.

Quality gates

> "A gate fails the build if new code drops below a standard — coverage, static analysis findings, or a security scan on dependencies. The one I would always want is a **dependency vulnerability scan**, because in BFSI a known CVE in a transitive dependency is a real audit finding, not a nice-to-have."

On coverage as a gate: it measures what was *executed*, not what was *verified* — a test with no assertion still counts. Useful as a floor, not as a target.

Deployment strategies — recognise the names

Strategy	How it works
Rolling	Replace instances gradually. The Kubernetes default.
Blue-green	Two identical environments; switch traffic over at once. Instant rollback, double the infrastructure.

Strategy	How it works
Canary	Send a small share of traffic to the new version first, watch the error rate, then widen.

The constraint they all share, and the one worth saying: for any of these, **two versions run at the same time for a while** — so database changes must be backwards compatible. Add a nullable column, deploy, then use it; remove it only in a later release. That is the **expand-then-contract** rule, and it is exactly what Liquibase-style versioned migrations are for.

Rollback

> "The first question before any release is what the rollback is, decided **before** deploying rather than during an incident. With Kubernetes that is `kubectl rollout undo`. The part that does not roll back cleanly is the database, which is why migrations need to be additive and backwards compatible — you can redeploy the old code, but you cannot un-drop a column."

Your honest material — use it

You genuinely have this, so say it:
- **SIT and UAT release cycles** — a real gated process, with QA validating before promotion.
- **Deployment validation** — checking the release actually worked in the environment rather than assuming it did.
- **Code reviews** as a gate before merge.
- **Production incident resolution** — analysing logs and execution plans, finding root cause, coordinating the fix with QA. That is the "support production releases" line in the JD.

12-factor ideas worth naming

Two of the twelve are enough, and both connect to things you know:
- **Configuration comes from the environment**, not from the artefact — so one build runs everywhere.
- **Services are stateless and disposable** — so any instance can be killed or replaced, which is what makes scaling and rolling updates possible. Same reason Spring beans are stateless and a JWT beats a server-side session.

✓ Check yourself before moving on

1. CI vs CD vs continuous deployment — and why BFSI usually gates the last one.
2. **Build once, promote the same artefact** — and why rebuilding per environment is wrong.
3. The pipeline stages in order.
4. Blue-green vs canary vs rolling, and the backwards-compatible migration rule they share.
5. Your boundary statement, said before you are asked for it.

11 · Coding Round

Time needed: 90 minutes. Write the code, do not read it.

Reading solutions feels productive and is nearly worthless. Open an IDE and type these from an empty file.

What to expect at Infosys

The coding bar for an experienced hire at a services company is **moderate**. They are not asking for dynamic programming. Expect:

- String and array manipulation
- Collections and HashMap usage
- Java 8 stream problems
- One "write a small class" or design question
- Sometimes SQL instead of code

They are checking: can you write correct, compiling Java; do you handle edge cases; do you talk while you work. Not whether you know a clever trick.

The process, which is half the marks:
1. **Restate the problem.** "So I need to return the first character that appears once, and null if there isn't one?"
2. **Ask about edge cases** *before* coding. Empty input? Nulls? Case sensitivity?
3. **Say your approach out loud.** "I'll count with a LinkedHashMap so I keep order, then find the first with a count of one."
4. **Then write it.**
5. **Trace one example** out loud when done.
6. **State the complexity.** They almost always ask, so beat them to it.

Problem 1 ● — First non-repeating character

The most common Java string question in Indian interviews.

```java
public static Character firstNonRepeating(String input) {
    if (input == null || input.isEmpty()) return null;

    Map<Character, Integer> counts = new LinkedHashMap<>();   // preserves insertion order
    for (char c : input.toCharArray()) {
        counts.merge(c, 1, Integer::sum);
    }

    for (Map.Entry<Character, Integer> e : counts.entrySet()) {
        if (e.getValue() == 1) return e.getKey();
    }
    return null;
}
```

Say: "LinkedHashMap, not HashMap, because I need insertion order to find the *first* one. Two passes, so O(n) time and O(k) space where k is the distinct character count."

Problem 2 ● — Reverse a string, and check a palindrome

```java
public static String reverse(String s) {
    if (s == null) return null;
    return new StringBuilder(s).reverse().toString();
}

// they often want it without the library method
public static String reverseManual(String s) {
    char[] chars = s.toCharArray();
    int left = 0, right = chars.length - 1;
    while (left < right) {
        char tmp = chars[left];
        chars[left++] = chars[right];
        chars[right--] = tmp;
    }
    return new String(chars);
}

public static boolean isPalindrome(String s) {
    String clean = s.replaceAll("[^A-Za-z0-9]", "").toLowerCase();
    int left = 0, right = clean.length() - 1;
    while (left < right) {
        if (clean.charAt(left++) != clean.charAt(right--)) return false;
    }
    return true;
}
```

The two-pointer palindrome is better than reversing and comparing, because it can exit early and uses no extra string. Say that.

Problem 3 ● — Duplicates in an array

```java
// find duplicates
public static Set<Integer> findDuplicates(int[] nums) {
    Set<Integer> seen = new HashSet<>();
    Set<Integer> dupes = new HashSet<>();
    for (int n : nums) {
        if (!seen.add(n)) dupes.add(n);   // add() returns false if already present
    }
    return dupes;
}

// remove duplicates, keeping order
public static List<Integer> removeDuplicates(List<Integer> nums) {
    return new ArrayList<>(new LinkedHashSet<>(nums));
}
```

`seen.add(n)` returning false is the neat part — one lookup instead of `contains` then `add`.

Problem 4 ● — Stream problems

These are very likely, because your resume says Java 8.

```java
// count word frequency
Map<String, Long> freq = Arrays.stream(sentence.toLowerCase().split("\\s+"))
        .collect(Collectors.groupingBy(w -> w, Collectors.counting()));

// second highest number
Optional<Integer> second = nums.stream()
        .distinct()
        .sorted(Comparator.reverseOrder())
        .skip(1)
        .findFirst();

// group employees by department
Map<String, List<Employee>> byDept = employees.stream()
        .collect(Collectors.groupingBy(Employee::getDepartment));

// highest paid employee per department
Map<String, Optional<Employee>> topPerDept = employees.stream()
        .collect(Collectors.groupingBy(Employee::getDepartment,
                Collectors.maxBy(Comparator.comparing(Employee::getSalary))));

// sum of even numbers
int sum = nums.stream().filter(n -> n % 2 == 0).mapToInt(Integer::intValue).sum();

// sort a map by value, descending
Map<String, Integer> sorted = map.entrySet().stream()
        .sorted(Map.Entry.<String, Integer>comparingByValue().reversed())
        .collect(Collectors.toMap(Map.Entry::getKey, Map.Entry::getValue,
                                  (a, b) -> a, LinkedHashMap::new));
```

That last one is worth practising — the four-argument `toMap` with `LinkedHashMap::new` is needed to keep the sort order, and people always forget it.

Problem 5 — Two sum

```java
public static int[] twoSum(int[] nums, int target) {
    Map<Integer, Integer> seen = new HashMap<>();     // value -> index
    for (int i = 0; i < nums.length; i++) {
        int need = target - nums[i];
        if (seen.containsKey(need)) return new int[]{ seen.get(need), i };
        seen.put(nums[i], i);
    }
    return new int[]{};
}
```

Say: "The brute force is two nested loops, O(n²). Storing what I have seen in a HashMap makes the lookup O(1), so it becomes one pass, O(n) time and O(n) space. That is the classic time-for-space trade."

Talking about the brute force *first*, then improving it, is exactly what they want.

Problem 6 — Fibonacci and factorial

```java
// iterative - preferred, no stack growth
public static long fib(int n) {
    if (n <= 1) return n;
    long prev = 0, curr = 1;
    for (int i = 2; i <= n; i++) {
        long next = prev + curr;
        prev = curr;
        curr = next;
    }
    return curr;
}

// recursive with memoisation, if they ask for recursion
public static long fibMemo(int n, Map<Integer, Long> memo) {
    if (n <= 1) return n;
    return memo.computeIfAbsent(n, k -> fibMemo(k - 1, memo) + fibMemo(k - 2, memo));
}
```

Naive recursion is $O(2^n)$ and will be called out. Memoisation makes it O(n).

Problem 7 — Reverse words, count vowels, anagram

```java
public static String reverseWords(String s) {
    String[] words = s.trim().split("\\s+");
    Collections.reverse(Arrays.asList(words));
    return String.join(" ", words);
}

public static boolean isAnagram(String a, String b) {
    if (a == null || b == null || a.length() != b.length()) return false;
    char[] x = a.toCharArray(), y = b.toCharArray();
    Arrays.sort(x); Arrays.sort(y);
    return Arrays.equals(x, y);
}
```

For anagram, mention the alternative: counting characters in a map is O(n) versus O(n log n) for sorting. Naming the trade-off scores.

Problem 8 — "Write a class" questions

Singleton, thread safe:

```java
public class ConfigManager {
    private static volatile ConfigManager instance;
    private ConfigManager() { }

    public static ConfigManager getInstance() {
        if (instance == null) {                          // first check, no lock
            synchronized (ConfigManager.class) {
                if (instance == null) {                  // second check, with lock
                    instance = new ConfigManager();
                }
            }
        }
        return instance;
    }
}
```

This is **double-checked locking**. `volatile` is essential — without it another thread can see a partially constructed object due to instruction reordering.

Say the better answer too: "In a Spring application I would not write this at all. A `@Component` is already a singleton in the container, and that is easier to test than a static instance. The enum singleton is also simpler if I truly need one outside Spring."

Immutable class:

```java
public final class Money {                       // final: cannot be subclassed
    private final BigDecimal amount;             // final fields
    private final String currency;

    public Money(BigDecimal amount, String currency) {
        this.amount = amount;
        this.currency = currency;
    }
    public BigDecimal getAmount() { return amount; }   // no setters
}
```

The four rules: make the class `final`, all fields `private final`, no setters, and if a field is mutable (a `Date`, a `List`) return a **defensive copy** from the getter rather than the field itself.

Java-specific traps they enjoy

"What does this print?"

```
Integer a = 127, b = 127;
Integer c = 128, d = 128;
System.out.println(a == b);   // true  - Integer cache covers -128..127
System.out.println(c == d);   // false - outside the cache, different objects
System.out.println(c.equals(d)); // true
```

```
String s1 = "abc";
String s2 = "abc";
String s3 = new String("abc");
s1 == s2          // true  - string pool
s1 == s3          // false
s1.equals(s3)     // true
s1 == s3.intern() // true  - intern() returns the pooled instance
```

```
System.out.println(0.1 + 0.2 == 0.3);   // false - floating point
// which is exactly why money uses BigDecimal
```

✓ Practice plan for this file

Type problems **1, 3, 4 and 5** from an empty file, no copy-paste. If you can write those four while explaining what you are doing, you can handle this round.

12 · Backend System Design

Time needed: 75 minutes

At this level Infosys will not ask you to design Netflix. But the JD is full of consulting language — *"problem definition, effort estimation, diagnosis, solution generation and design"* — so expect one design question, and expect them to be watching **how you think**, not whether you produce a perfect architecture.

The framework — use it for every design question

```
1. CLARIFY      Ask 2-3 questions. Never start designing immediately.
2. SCOPE        State the core use cases you will design for.
3. API          What endpoints, what request and response shapes.
4. DATA MODEL   Tables or documents, and the key relationships.
5. FLOW         Walk one request end to end.
6. SCALE        What breaks first, and what you would do about it.
```

Step 1 is the one candidates skip, and it is the one the JD explicitly asks for. "Problem definition" is literally in the job description.

Clarifying questions that fit almost anything:
- "Roughly what volume are we designing for — hundreds a day or millions?"
- "Is this read-heavy or write-heavy?"
- "Does this need to be strongly consistent, or is eventual consistency acceptable?"
- "Is there an existing system I am integrating with, or is this greenfield?"

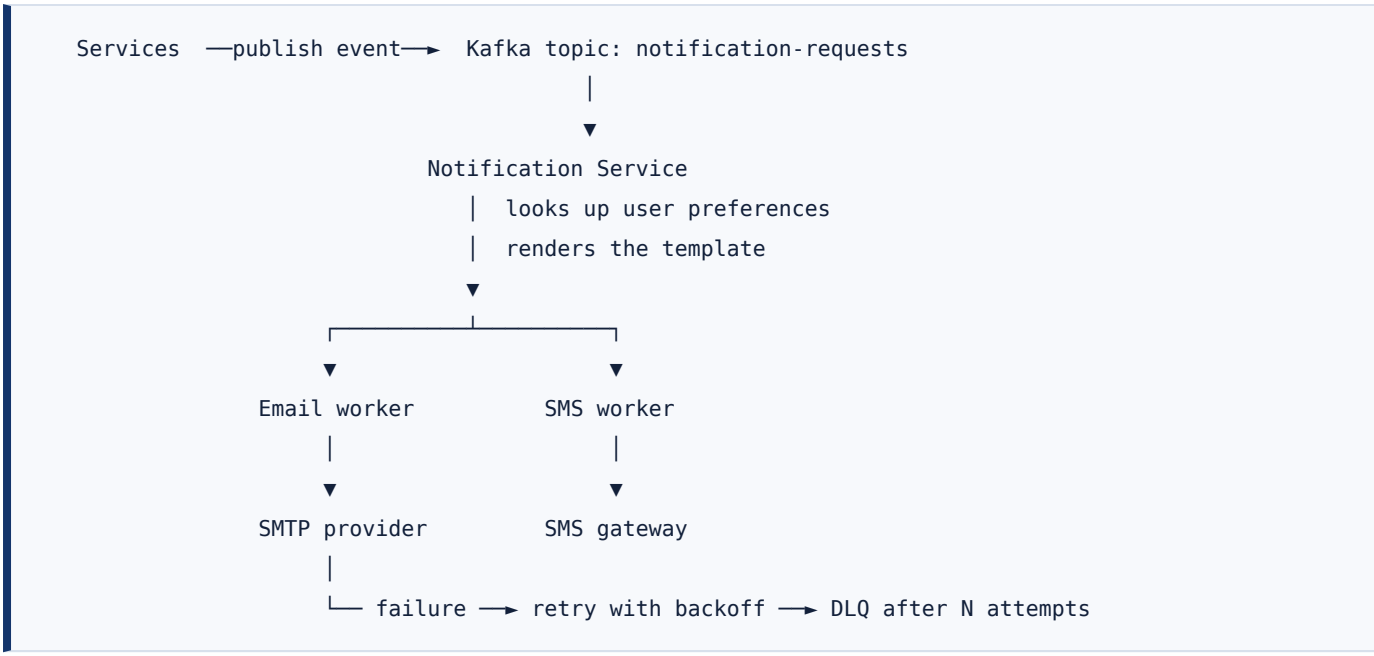
That third question is a strong one to ask in a banking context, because the answer is usually "strongly consistent for money, eventually consistent for notifications" — and saying that shows domain judgement.

Design 1 ● — A notification service

Most likely for you, because your resume already describes half of it.

Clarify: Which channels — email, SMS, push? Do we need delivery status? Are there priorities, or is everything equal? Roughly what volume?

The design:



The points to make:
- **Asynchronous, always.** The calling service must never wait for an email. This is exactly what you did with RabbitMQ.
- **Preferences and templates** live in the notification service, not scattered across callers. One place to change how something is worded.
- **Retry with exponential backoff**, then a dead letter queue, then an alert. Never silently drop a notification.
- **Idempotency**: a duplicate event must not send two emails. Key on an event ID and record what has been sent.
- **Delivery status** written back so support can answer "did the customer get it?"

Then land it: *"This is close to what I built — the async part with RabbitMQ for emails, and Kafka for publishing state changes to several consumers."*

Design 2 ● — A transaction recall / approval workflow

Your actual domain. If they let you pick, pick this.

Clarify: Who can raise a recall, who approves? Is there a time limit? Does it need an audit trail? Single or multi-level approval?

Data model:

```
recall_request
  id, transaction_id, requested_by, reason, status, created_at, updated_at

recall_approval
  id, recall_request_id, approver_id, decision, comments, decided_at

audit_log
  id, entity_type, entity_id, action, actor, before_state, after_state, timestamp
```

The state machine — draw it, it always impresses:

```
DRAFT → SUBMITTED → UNDER_REVIEW → APPROVED → COMPLETED
                                 │
                                 └──────→ REJECTED
```

The points to make:
- **Model the states explicitly** and validate transitions in one place. An approved request must never go back to draft. *(Your resume mentions unit tests for state-transition validation — say that here.)*
- **Authorisation server side.** A requester must not be able to approve their own request. Enforce in the service layer, not the UI.
- **Append-only audit log.** In banking every state change must be attributable — who, what, when, and what it was before. Never update audit rows.
- **Publish each state change as an event** for compliance and reporting consumers. That is your Kafka work.
- **Idempotency on submit**, so a double-click does not create two recall requests.

Design 3 — A URL shortener

The classic warm-up. Know it in case it comes up.

Core: `POST /shorten` takes a long URL, returns a short code. `GET /{code}` returns a **301 or 302 redirect**.

Generating the code: a base62 encoding of an auto-increment ID is simple and collision-free. Hashing the URL is an alternative but needs collision handling.

Scale points: it is extremely read-heavy, so cache the code-to-URL mapping in Redis in front of the database. The lookup must be a primary key or unique index hit. Mention 301 (permanent, browser caches it, so you lose click analytics) versus 302 (temporary, every click reaches you) — that trade-off is the interesting part.

Design 4 — "How would you design a REST API for X?"

The generic answer, which is worth having ready:

```
GET    /api/v1/transactions          list, with pagination and filters
GET    /api/v1/transactions/{id}     one
POST   /api/v1/transactions          create      → 201 + Location header
PUT    /api/v1/transactions/{id}     full replace
PATCH  /api/v1/transactions/{id}     partial update
DELETE /api/v1/transactions/{id}     remove      → 204
```

The rules to state:
- **Nouns, not verbs.** `/transactions/{id}/recall` as a sub-resource, not `/recallTransaction?id=`.
- **Correct status codes.** 201 for created, 204 for no content, 400 for bad input, 401 vs 403, 404, 409 for a conflict, 422 for validation, 500 for server errors.

- **Pagination on every list endpoint.** Never return an unbounded collection.
- **Versioning** in the URL, and only additive changes within a version.
- **A consistent error shape** across every endpoint, via `@RestControllerAdvice`.
- **Idempotency** for non-safe operations, usually via a client-supplied key.
- **DTOs, never entities, at the boundary.** Returning entities leaks your schema, causes lazy-loading surprises during serialisation, and means a database change breaks your API contract.

That last point is a genuinely good one and few candidates say it.

Design 5 — "This API is slow. What do you do?"

You have lived this. Give a process, not a list of fixes.

```
1. MEASURE     Where is the time actually going? Application logs and timings first,
               then the slow query log. Do not guess.

2. CATEGORISE  Is it the database, the application, or an external call?
               - Database  → N+1, missing index, bad JOIN
               - Application → inefficient loop, unnecessary object creation
               - External  → a synchronous call that should be async

3. FIX THE BIGGEST ONE FIRST

4. RE-MEASURE  Prove it. Compare before and after.
```

Then make it concrete: *"That is exactly the process that took the report from 15 seconds to 6. The measurement step told me it was N+1 rather than the query itself, which I would not have guessed."*

The honest line that scores: *"My first assumption about the cause was wrong, which is why I measure before optimising."*

✓ Check yourself before moving on

1. Recite the six-step framework.
2. Design the notification service out loud in 3 minutes, including retries and DLQ.
3. Draw the recall state machine and say why the audit log is append-only.
4. Give five REST API design rules, including DTOs at the boundary.

13 · Talking About Your Experience

Time needed: 60 minutes, said out loud

First, the context: your real position

You are stronger for this role than you probably feel, for three specific reasons.

1. You have real numbers. 15 seconds to 6 seconds. 2–3 seconds to under 1 second. Most candidates say "I optimised the API." You can say what was wrong, what you changed, and what it became. In a services company interview that is rare and it is remembered.

2. You have banking domain experience. The JD asks for "one or two industry domain knowledge". You worked on a UK private bank's transaction recall and card management systems. Infosys staffs projects by domain, and someone who already understands banking workflows is easier to place. Easier to place means easier to hire.

3. You used Kafka and RabbitMQ for different reasons. Not just "we had messaging" — a task queue for one job and an event log for another. That is a design decision, and it is the most senior-sounding thing on your resume.

What to watch: your experience is about 1 year 6 months of engineering. Do not oversell it (see section 5), and do not undersell it either. Depth on the things you actually did beats vague claims about things you did not.

• "Tell me about yourself" — your script

Structure: **Now → What you have built → Why this role.** Under two minutes.

> "I am a backend developer with close to two years of experience, currently at Zensar working on banking systems for C. Hoare & Co, a private bank in the UK.
>
> My work is Java and Spring Boot with Spring Data JPA and MySQL, and I have built two platforms there: a transaction recall system and a business card management system. Both were REST APIs with role-based access control, and both had performance problems when I picked them up — I took report generation from 15 seconds to 6, and the card APIs from 2 to 3 seconds down to under a second, mostly by removing N+1 queries and indexing properly.
>
> The part I enjoyed most was moving work off the request thread. I replaced synchronous email sending with RabbitMQ so bulk approvals stopped timing out, and used Kafka to publish transaction state changes to compliance and reporting consumers.
>
> I have also built a microservices project of my own with Eureka and an API Gateway, which is why this role interested me — it is Java, Spring Boot and microservices, which is exactly the direction I have been moving in."

Practise this twice today and twice on the morning of the 5th. Time it.

Your four stories, prepared properly

For each: **what it was → what you decided → what was hard → what you would change.** The fourth is what makes an answer sound senior.

Story 1 ● — The report that took 15 seconds *(lead with this)*

> "Report generation on the recall platform was taking about 15 seconds, and users were complaining. My first assumption was that the query itself was heavy, but I turned on SQL logging and saw the same query repeating once per row — a classic N+1, because the report was walking each transaction's related entities.
>
> I replaced it with a projection query that joined and selected only the columns the report needed, so it stopped loading full entities. Then I looked at the execution plans, added indexes on the columns used in the WHERE and JOIN clauses, and rewrote a couple of JOINs that were forcing full scans. That took it from 15 seconds to 6.
>
> What I took from it is to measure first. My initial guess was wrong, and if I had acted on it I would have spent a day tuning a query that was not the problem."

Follow-ups to be ready for: *What is N+1?* (file 04) *How did you know which index to add?* (`EXPLAIN`, file 08) *Why 6 seconds and not 1?* — be honest: some of it was the sheer data volume and the report's scope, and the next step would have been caching or pre-aggregating.

Story 2 ● — The APIs that timed out on bulk approvals

> "On the business card system, approvals were sending emails synchronously inside the request. A single approval was slow, and bulk approvals were timing out — which was worse than slow, because the approval had actually succeeded but the user saw a failure.
>
> I moved the email to a RabbitMQ message and a separate consumer. The API went back to responding in milliseconds, and a mail server problem stopped being an API problem — the messages just wait in the queue.
>
> Separately the same APIs had N+1 issues, so I fixed those and set the right associations to lazy, which brought response times from 2 to 3 seconds down to under a second."

The reasoning to make explicit: the email is not something the user is waiting for, so it does not belong in the request.

Story 3 — Kafka for transaction state changes

Use this when asked about architecture or about Kafka. The key point is *why* Kafka and not a queue — the full answer is in file 06, Q1.

> "Transaction state changes needed to reach more than one consumer — compliance logging and report generation, with more likely later. With a queue the first consumer takes the message and the others never see it. With Kafka each consumer group reads the same events independently at its own pace, the events are retained so a consumer that was down can catch up, and compliance can replay history if it needs to rebuild."

Story 4 — A production incident

Your resume says you resolved production incidents by analysing execution plans and logs. Have one concrete example ready, in STAR form: what broke, how you found it, what you did, what happened, and what changed afterwards so it would not recur.

The part that matters most is the last one. "We added an alert / added an index / added a test so it would be caught earlier" turns a firefighting story into a process-improvement story — which is exactly the JD's *"assess current processes, identify improvement areas"*.

Your Classifieds project

Covered in file 05, Q7. Keep it brief unless they ask — your professional work is stronger. Its value is showing that you have designed a microservices system end to end rather than only worked inside one service.

●● Section 5 — The three questions your resume creates for THIS role

Prepare all three. None is disqualifying. Being caught without an answer is what costs you.

1. ●● The experience band — the JD says 3–6 years

Your dates:

```
Feb 2024 – Dec 2024   Talent Acquisition Executive, Artifint     ~10 months
Dec 2024 – Jun 2026   Software Engineer (Backend), Zensar        ~18 months
                      total professional experience             ~2 years 4 months
                      of which backend engineering               ~1 year 6 months
```

You are under the stated band. Face it calmly rather than hoping it does not come up.

First, the reassurance: you have the interview. Someone looked at your profile, saw the dates, and still wanted to talk. Experience ranges in postings are sourcing guidance, not a hard gate, and services companies hire across bands when the evidence is strong.

Second, and this matters more here than anywhere: NCG is a services company placing people into a client's GCC. **Your rate and your level are set from verified dates.** Background

verification against payslips and employment letters is routine. So **never round up** — being caught costs the offer at the worst possible moment, after you have resigned somewhere.

The answer, if it is raised directly:

> "In engineering, from December 2024 to June 2026 — about a year and a half — and around two and a half years of professional experience in total. I know that is below the range on the posting.
>
> What I would say is that it has been dense for the time. I have worked on a UK bank's platforms end to end — REST APIs with Spring Boot and JPA, event-driven work with both Kafka and RabbitMQ, Spring Security with role-based access control — and I have real performance results: report generation from 15 seconds to 6, and API response from 2–3 seconds to under one. I have also handled production incidents and root cause analysis. I would rather be judged on that than on the number."

Do not volunteer it if they do not raise it. Do not dodge it if they do. And do not apologise — say it the way you would state any other fact.

2. ●● The gap — you left in June

Your resume shows **Dec 2024 – June 2026**, and it is now **mid-November**. That is roughly **five months**, and it is the first thing an interviewer notices.

Two or three sentences. Factual. Calm. No apology.

> "[The real reason — the project rolled off / the engagement ended / a personal reason.] Since then I have been [specifically what you have done]. I was deliberate about wanting a backend role in financial services rather than taking the next thing available, which is why this one interested me."

Fill in the truth. Do not invent anything. Project roll-offs are routine in the services industry and no interviewer holds one against you.

At five months, "I was upskilling" is not enough — be specific, and this is where the weekend's work pays off twice:
- If you have been learning **Docker, Kubernetes or cloud**, say so — it is exactly what this JD asks for and what you are closing this weekend. *"I have been working through containerisation and deployment, which was the part of my last role I was furthest from."* That is honest, verifiable by the conversation you are about to have, and turns the gap into initiative.
- Anything on GitHub. Any certification.

Then close on availability: you can join immediately, which for a staffing-led role with a client waiting is a genuine advantage. Say it.

3. ● "Why recruitment, and then development?"

It is in the middle of your resume, so it will be asked. Answer directly, then turn it.

> "I finished my B.Tech in Computer Science in 2022, so development was always the plan, and I did the JSPIDERS Java full stack programme to build the skills properly. The Artifint role was **technical** recruitment — I was screening and coordinating interviews for engineering positions, so I was close to the work I wanted to do. [The real reason it happened.] I moved into engineering at Zensar in December 2024.
>
> That year was more useful than it sounds, honestly. I spent it talking to engineers and clients every day, and I am comfortable in requirement and client conversations in a way that is not always true of people who came straight through."

Read the JD again: *"close collaboration with frontend, QA, and business stakeholders"*, *"client-facing environment"*, *"collaborate with... business stakeholders to translate requirements into technical solutions."* This is a **client-facing GCC role**. You have a year of evidence that you can hold a professional conversation with a stranger about technical work. Almost nobody else in the queue can say that. **Do not present it as a detour — present it as why you are comfortable with the client-facing half of this job.**

Questions about your numbers

If they ask how you measured 15s → 6s, or 2–3s → under 1s, be ready with: application timings and logs before and after, plus the query counts from SQL logging. If the numbers came from someone else's measurements or a monitoring dashboard, say so — attributing accurately is better than being vague.

✓ Before the interview

1. Say the "tell me about yourself" script out loud, twice. Time it.
2. Write down the **real one-sentence reason** for the 2022–2023 gap, and rehearse the recruitment-to-development answer plus the "less than 3 years" answer. Say all three out loud until they sound calm rather than defensive.
3. Decide how you will state your experience duration, and make the resume match.
4. Be able to say every number without hesitating: **15→6, 2–3→under 1, 3 services, 200+ profiles**.
5. Have one production-incident story with a **what changed afterwards** ending.

14 · HR, Managerial and the Offer Conversation

Time needed: 60 minutes. Say every answer out loud.

Infosys usually runs a separate managerial or HR round, and it is not a formality. People clear the technical round and lose the offer here.

---

1. "Why Infosys?"

Do not say "it is a big company" or "for growth". Say something only someone who read the posting could say.

> "Three reasons.
>
> The role is Java, Spring Boot and microservices, which is exactly what I have been doing and where I want to go deeper. That is a straightforward fit.
>
> Second, I have banking domain experience from working on a UK private bank's systems, and Infosys works across financial services. I would rather build on a domain I already understand than start from nothing.
>
> Third, the job description talks a lot about working with clients — requirement analysis, seeking clarifications, proposing alternatives. I have done that at Zensar with a client team in another country, and before that I spent a year in technical recruitment, so I am comfortable in those conversations. It is the part of the work I actually enjoy, and it seems central to how Infosys operates."

That third point is your real differentiator. Use it.

2. "Why are you leaving Zensar?"

Never criticise your current employer. Frame it as moving toward something.

> "I have had good exposure at Zensar — I have owned features end to end on two banking platforms and worked across the full SDLC. What I am looking for now is more depth in microservices architecture rather than mostly working within one application, and more direct client-facing work. This role is explicitly microservices and consulting oriented, which is why I applied."

If asked about compensation, it is acceptable to say it is *a* factor, but never the first one you mention.

3. The JD's own language — prepare these four

The posting is full of consulting phrases. Each one is a question waiting to be asked.

"How do you handle unclear or incomplete requirements?" *(the JD's "problem definition" and "seek clarifications")*

> "I do not start building on an assumption. I write down what I have understood, including the gaps, and take it back with specific questions rather than 'please clarify'. On the recall workflow there were states where it was not defined who could act, so I listed the cases, proposed a default for each, and asked them to confirm. That is much faster to answer than an open question, and it gets a decision recorded."

"How do you estimate effort?" *(the JD's "effort estimation")*

> "I break the work into pieces small enough to reason about — API changes, data model changes, testing, integration — and estimate those rather than the feature as a whole. I add time explicitly for testing and for integration issues, because those are what people forget. And I flag what the estimate assumes, so if an assumption turns out to be wrong the estimate can be revisited rather than just being missed."

"How do you learn a new technology?" *(the JD's "research, literature surveys, POCs")*

> "Official documentation first, because tutorials teach you one path and the docs tell you the model. Then I build a small proof of concept, because reading does not surface the problems. When I picked up Kafka I built a small producer and consumer before touching the real system, which is where I actually understood consumer groups and offsets. Then I check how it is used in the existing codebase, since conventions there matter more than my preferences."

"How do you approach diagnosing a production issue?" *(the JD's "diagnose the root cause")*
Use your real process from file 10, Design 5: reproduce if possible, check logs and timings, form a hypothesis, verify it before fixing, fix, then add something that would have caught it earlier. Finish with a concrete example.

4. Standard behavioural questions — use STAR

Situation → **T**ask → **A**ction → **R**esult. Under 90 seconds. Always end on a result.

"Tell me about a challenging technical problem." → The 15s → 6s story (file 11).

"A time you disagreed with someone." → Best if it is a technical disagreement you resolved with evidence rather than opinion. The pattern that works: you raised the concern, then brought data or a working alternative instead of just objecting.

"A time you made a mistake." → Have one. Refusing to answer this looks worse than the mistake. Pick something real, own it plainly, and finish with what you changed. A missed edge case caught in QA, with "and I added a test for that state transition afterwards", is a good shape.

"How do you handle pressure or a tight deadline?" → Scope, communicate early, do not silently slip. You worked in SIT and UAT cycles, so you know what happens when something half-finished reaches testing.

"What is your weakness?" → Real, bounded, with the fix.

> "Coming from a non-linear route into development, I was initially slower at reading large unfamiliar codebases than people who had been doing it longer. I have got much better by tracing one request end to end through the code before changing anything, rather than jumping straight to the file I think is responsible."

Never "I am a perfectionist" or "I work too hard".

"Where do you see yourself in 3 years?"

> "Owning a service or a module end to end, and being the person on the team who is trusted with performance and data-layer problems. Longer term I would like to move toward solution design, which is part of why the consulting side of this role appeals."

"How do you work in a team?" → Mention code reviews (you do them), QA coordination (on your resume), and being specific in reviews rather than vague.

5. Logistics questions — answer instantly

Question	How to answer
Notice period	State it exactly. If it is 60 or 90 days, say so, and mention whether buyout is possible. Do not guess.
Willing to relocate?	Your resume already says "open to relocate anywhere in India / remote", so **say it clearly and early** — it is a real advantage with Infosys, which staffs across Hyderabad, Bangalore, Pune, Chennai and Mysore. Just make sure it is actually true, because they will hold you to it.
Work from office?	Infosys is largely back in office. Answer directly.
Shifts / on-call?	Client projects sometimes need overlap with UK or US hours. You already work with a UK client, so say that — you know what it involves.
Do you have other offers?	Honest and brief. Do not invent leverage; it usually backfires.

⚠ If any of these are genuinely a problem, say so **now**, politely. Accepting and renegotiating later damages the relationship and your reference.

6. Salary

Know your numbers before the call: your current fixed CTC, your current total CTC, and the number you want. Do not do arithmetic live.

When asked for expectations:

> "My current CTC is [X]. Based on my experience with Java, Spring Boot, microservices and messaging, and my banking domain background, I am looking for [Y]. But I am interested in the role and the project, so if the fit is right I am open to discussing it."

Rules:
- **Never lie about your current CTC.** Infosys verifies with payslips and Form 16. Being caught costs the offer, usually after you have resigned elsewhere.
- **Ask for the breakup** at offer stage: fixed, variable, retention bonus, and what the in-hand figure is. Large companies often put a meaningful share in variable pay.
- **Ask about the band or level.** Infosys has defined levels, and the band affects your future increments more than the starting number does.
- If they ask first, it is completely normal to ask **"what range have you budgeted for this role and level?"**

7. 🔴 Questions YOU ask them

Have four ready, ask two or three. Ending with "no, I'm good" reads as no interest.

Technical:
1. "Which domain would this role sit in — is it financial services, given the skills?"
2. "Is the project greenfield microservices, or modernising an existing monolith?"
3. "What does the current stack look like — Spring Boot version, cloud platform, CI/CD?"
4. "How large is the team, and would I own a service end to end?"

Role and growth:
5. "What would a successful first three months look like?"
6. "Is there scope to move toward design and client-facing work over time?"
7. "How does Infosys handle certification and upskilling for someone at this level?"

The one that makes you memorable, for the managerial round:

> "What is the biggest technical problem the team is dealing with right now that this hire would help with?"

Then **listen**, and connect your answer to it. That is how you close.

8. On the day

- Two printed copies of your resume, and a copy open on your laptop.
- **Know your own resume cold.** Everything on it is fair game — Kafka, RabbitMQ, Eureka, JWT, Hibernate lazy loading, MongoDB, execution plans. If you touched something briefly, say so honestly rather than overstating.
- Carry your ID, and have your employment and education documents ready — Infosys onboarding is document heavy.

- If it is online: join five minutes early, test camera and microphone, sit somewhere quiet with a plain background, and keep an offline copy of your notes in case the network drops.
- Dress smart. Slightly overdressed beats underdressed.

9. If you get stuck

- **Do not freeze — narrate.** "Let me think about that for a second" is a complete and acceptable sentence.
- **Do not bluff.** "I have not used that directly. My understanding is X — is that the direction you mean?" has never cost anyone an offer. Bluffing has, because the next question exposes it.
- **Ask what they are getting at.** "Do you mean at the database level or in the application?"
- If you fumble one question, let it go. People lose interviews by replaying question 4 while answering question 7.

15 · Delivery, Code Review and Working With Clients

Time needed: 60 minutes

First, the context: read this job description again

Almost the entire responsibilities section of this posting is about things that are **not writing code**:

> *"interface with the client for quality assurance, issue resolution and ensuring high customer satisfaction... understand requirements, create and review designs, validate the architecture... participate in project estimation, provide inputs for solution delivery, conduct technical risk planning, perform code reviews and unit test plan reviews... lead and guide your teams towards developing optimized high quality code deliverables, continual knowledge management and adherence to the organizational guidelines and processes."*

The only technical requirement listed is **Java and Spring Boot**, as both primary and preferred skill. Everything else in that block is about **how you deliver**.

What that tells you: this is not a pure coding role. It is Infosys's analyst-shaped template — someone who can be put in front of a client, review other people's work, and be trusted to follow process. The technical round will still test Java and Spring Boot properly, but there will be a second conversation about all of the above, and most candidates walk into it with nothing prepared.

The good news: your resume already contains evidence for nearly every item. Code reviews, QA coordination, requirement analysis, deployment validation, production incident resolution. You have done these things. What you need is to have the answers **ready**, and to describe them as ownership rather than participation.

⚠ **One framing note.** You have under two years of experience, so do not claim to have led a team. Claim what is true: you reviewed code, you coordinated with QA, you handled production issues. Then show you understand what the next level up involves. Interviewers respect that far more than an inflated claim they can puncture with one follow-up.

Q1. ● "Have you done code reviews? What do you look for?"

Near-certain, because the JD names it explicitly.

Say this.

> "Yes, code reviews were part of our normal process — both getting mine reviewed and reviewing others'.
>
> I look in a rough order of importance. First **correctness**: does it do what the requirement says, and are the edge cases handled — nulls, empty results, boundary values. Second **error handling**: are exceptions handled at the right level, or is something being swallowed silently, which is the worst outcome because the failure becomes invisible.
>
> Then **the data layer**, because that is where I have found the most real problems. Is there an N+1 sitting in a loop, is a query missing an index it will need, is `@Transactional` on the right method. Then **tests** — not the count, but whether the meaningful cases are covered. And last **readability**: naming, method length, duplication. That matters, but I would not block a change on it alone."

The follow-up: "How do you give the feedback?" This is really a communication question.

> "I separate what must change from what is a suggestion, and say which is which — a comment that just says 'this could be better' wastes everyone's time. I ask rather than instruct when I am not sure of the context, because often there is a reason I cannot see. And if a thread goes past two or three replies I stop typing and talk to the person, because that usually means we are misunderstanding each other rather than disagreeing."

Real-time example. In a card management change, a reviewer's comment on my code was that a status update was not inside the transaction boundary, so a failure partway through would have left the record updated but the audit row missing. That was a correctness bug a test would not have caught, and it is the kind of thing I look for now when I review someone else's.

Q2. ● "What is a unit test plan review? What makes a test plan good?"

Explicitly in the JD, and most candidates have never thought about it.

Say this.

> "A test plan review is checking that the tests cover the right things **before** the code is written or signed off, rather than discovering the gap in UAT.
>
> What I look for is whether the tests cover **behaviour and rules**, not just lines. A suite with high coverage that only tests the happy path is worse than a smaller suite that tests the rules that actually matter. So I check three things: is the happy path covered, are the **failure and boundary cases** covered, and are the **state transitions** covered — the transitions that must be **prevented** as much as the ones that are allowed."

Real-time example — this is directly on your resume.

> "On the business card workflow I wrote unit tests for state transition validation. The important tests were not 'an approval succeeds' — they were the negative ones: an already approved request cannot be approved again, a rejected request cannot move back to pending, and a requester cannot approve their own request. Those are the rules that cost money if they break, and they are exactly what a test plan review should be checking for."

If they ask about coverage targets:

> "Coverage tells you what was executed, not what was verified. A test with no assertion still counts towards it. I would rather aim for the critical paths being genuinely tested than chase a percentage, though a very low number is still a useful warning sign."

Q3. ● "How do you estimate effort?"

In the JD as "participate in project estimation". Give a method, not a number.

Say this.

> "I break the work down until the pieces are small enough that I can actually picture doing them — API changes, data model changes, the tests, and the integration. Estimating a whole feature in one number is guessing; estimating six small pieces is much closer.
>
> I explicitly include the things people leave out: testing, code review turnaround, and integration issues. Integration is where estimates fail most often, because it is the part that depends on other people.
>
> And I state the assumptions with the estimate. If I have assumed the API contract is fixed and it then changes, the estimate should be revisited rather than just missed — stating the assumption is what makes that conversation possible instead of it looking like a slip."

Real-time example. A change that looks like one field on a screen is rarely one field: it is the DTO, the validation, the entity and migration, the query, the tests, and the downstream event if anything consumes it. Walking that list is how I avoid estimating the visible part only.

If they ask about techniques: mention that you have worked in Agile with story points and relative sizing, and that the value of a group estimate is mostly in surfacing the different assumptions people are holding, not in the number itself.

Q4. ● "What is technical risk planning? Give me an example."

The JD phrase is "conduct technical risk planning". Most people freeze on this. It is simpler than it sounds: **what could go wrong, how likely, how bad, and what we do about it.**

Say this.

> "It is identifying what could go wrong technically before it does, and deciding on the response in advance rather than during the incident.
>
> On a change I would ask: what does this depend on that is outside my control; what happens under a load much higher than we test with; what is the blast radius if it fails; and how would we roll it back. Then for the serious ones, either mitigate it in the design or agree a fallback."

Real-time example — use your own migration.

> "When we moved email notifications from synchronous sending to RabbitMQ, the obvious risk was that the queue or consumer goes down and notifications silently stop — which is worse than the timeout we were fixing, because a timeout is visible and silence is not. So the mitigations were retries with backoff, a dead letter queue so a failing message moves aside instead of blocking the queue, and alerting on the DLQ so somebody actually finds out. Asynchronous processing without a DLQ and monitoring just moves the failure somewhere nobody is looking."

That answer is strong because the risk is real, and the mitigations are specific.

Q5. ● "How do you review a design, or validate an architecture?"

The JD says "create and review designs, validate the architecture". You are not expected to be an architect. You are expected to ask sensible questions.

Say this.

> "I check it against the requirement first — does the design actually do what was asked, and are the non-functional parts stated: expected volume, response time, what has to be consistent versus what can be eventual.
>
> Then the practical questions. Where does the data live and who owns it. What happens when a dependency is unavailable. Is anything a single point of failure. How is it secured, and is authorisation enforced server side. How will we know it is working in production — logging, correlation IDs, metrics. And how does it get released and rolled back.
>
> The last one I ask is whether it is more complex than the problem requires. Simplicity is a real design property, and complexity that is not paying for itself is a cost forever."

Real-time example. Publishing transaction state changes to Kafka rather than calling the compliance and reporting services directly was a design decision with a trade-off: it removed the coupling and let each consumer read at its own pace, at the cost of eventual consistency and a harder debugging story. Being able to state the cost of your own design, not just its benefit, is what a design review is for.

Q6. ● "How do you handle a client raising a quality issue?"

The JD's first line is client interfacing for quality assurance and issue resolution. You worked with a UK bank, so you have real material.

Say this.

> "First, acknowledge quickly and separate the two things: what we do to contain it now, and what caused it. Clients get much more anxious about silence than about the problem itself.
>
> Then get to the actual facts before promising anything — reproduce it, check the logs and what changed recently. I do not commit to a cause or a timeline until I know, but I do commit to when I will come back with an update, and I keep that.
>
> Then fix, verify, and say plainly what happened and what we have changed so it does not recur. That last part is what actually restores confidence."

Real-time example. Production incidents I worked on were diagnosed by analysing SQL execution plans and application logs to find the root cause, then coordinating the fix with QA before it went out. The important part was not the fix, it was that QA validated it in SIT before it reached the client again.

The line that lands with a services company:

> "Being wrong quickly and openly costs far less than being confident and wrong slowly."

Q6b. ● Maven and Gradle — the JD names both

You use Maven. Know the lifecycle, and know what Gradle is.

The Maven lifecycle, in order — this is the question:
`validate → compile → test → package → verify → install → deploy`

- `mvn clean install` — the one everyone runs. Wipes `target/`, builds, runs tests, and puts the artefact in your **local** repository so other local projects can use it.
- `mvn package` — builds the JAR but does not install it locally.
- `mvn clean verify` — build plus tests plus integration tests, without installing. The right one in a CI pipeline.

Scopes: `compile` (default), `provided` (available when compiling, supplied at runtime), `runtime`, `test` (JUnit, Mockito).

Two things worth volunteering, because they signal real build experience:
- **The parent POM.** `spring-boot-starter-parent` manages versions, which is why you add `spring-boot-starter-web` with no version and get a compatible set.
- **Dependency conflicts.** Maven resolves to the **nearest** version in the tree, which is how you end up running a version you never asked for. `mvn dependency:tree` finds it; `<dependencyManagement>` or an exclusion pins it.

Maven vs Gradle:

> "Maven is XML and convention-driven — predictable, verbose, and everyone can read it. Gradle uses a Groovy or Kotlin DSL, so it is more flexible, and it is faster on large builds because of incremental builds and a build cache. I have used Maven; Gradle expresses the same ideas, so picking it up is a matter of syntax rather than concepts."

Q7. "How do you guide or mentor someone more junior?"

Answer honestly at your level.

Say this.

> "I have not formally led a team, so I will answer about what I have actually done. Most of it has been through code review and pairing on problems. What I have learned is that pointing at the answer does not stick — explaining why, or asking the question that gets them there, does.
>
> The other thing is writing things down. If I explain the same thing twice, that is a documentation gap, not a person problem."

That answer is credible, and the honesty at the start makes the rest believable.

Q8. "Knowledge management" and "adherence to processes" — what do these mean?

Both are literally in the JD, and both sound like filler until you answer them concretely.

Knowledge management — say this.

> "Making sure what I know is not only in my head. In practice that is API documentation that is actually current — we used Swagger, so it lives with the code rather than drifting in a separate document — plus runbooks for things like what to check when the queue backs up, and proper handover notes when someone rotates off. The test is whether someone else could pick up my work if I were away, and I would rather find that out deliberately than during an emergency."

Adherence to processes — say this.

> "In a bank the process exists for a reason, and the reason is usually that something went wrong once. We had a defined SDLC — requirement analysis, design, implementation, unit tests, then SIT and UAT before release — with code review and deployment validation as gates. I have never found the process to be the bottleneck; what slows things down is ambiguity in requirements, which is why I push to get those pinned down early."

Q9. "How do you handle unclear or changing requirements?"

Say this.

> "I do not start building on an assumption I have not confirmed. I write down what I have understood, including what is missing, and go back with **specific** questions rather than asking them to clarify — a specific question with a proposed default is much faster for someone to answer, and it gets a decision on record.
>
> When requirements change mid-work, the important thing is to say immediately what it means for the estimate and what else it affects, rather than quietly absorbing it. Absorbing changes silently is how a project ends up late with nobody able to explain why."

Real-time example. On the recall workflow there were states where it was not defined who was allowed to act. Rather than pick something, I listed the cases, proposed a default for each, and asked for confirmation. That took a short call instead of a long email thread, and it meant the rules were written down rather than living in the code.

Q10. "What makes code high quality?" *(the JD says "optimized high quality code deliverables")*

Say this.

> "Four things, roughly in order.
>
> It is **correct**, including the cases that are not the happy path. It is **readable**, because it will be read far more often than it is written, and the next person may be me in six months. It is **tested** where it matters, particularly the rules that cost money if they break. And it **performs acceptably under real data volumes**, not just the ten rows in a developer's local database — which is the specific mistake behind both of the performance problems I fixed, because both were fine in testing and slow in production.
>
> The one I would add for a bank is **traceability**. If something goes wrong, can we tell what happened, who did it and when. That is why proper logging and an audit trail are part of the deliverable, not an extra."

✓ Check yourself before moving on

1. What you look for in a code review, **in order**, and how you give the feedback.
2. What makes a good unit test plan — and the negative state-transition example.
3. Your estimation method, including stating assumptions.
4. The RabbitMQ risk-planning example, with the DLQ and alerting.
5. How you handle a client-raised quality issue, in four steps.

16 · Final Cheat Sheet — INTERVIEW MORNING ONLY

45 minutes. Nothing else. No new material.
If a line here does not ring a bell, open that file for three minutes and come back.

The 8 answers that carry this interview

1. **HashMap:** *"`hashCode()`, then a spread of `h ^ (h >>> 16)`, then bucket index `(n-1) & hash`. Collisions chain in a linked list, and since Java 8 a bucket over 8 entries becomes a red-black tree. Default capacity 16, load factor 0.75, so it resizes and rehashes at 12."*
2. **equals/hashCode:** *"Equal objects must have equal hash codes. Override `equals` without `hashCode` and the map looks in the wrong bucket and never finds the entry."*
3. **N+1:** *"One query for the list, then one per row for the relation. Fix with JOIN FETCH, `@EntityGraph`, batch fetching, or a DTO projection. That is what took my report from 15 seconds to 6."*
4. `@Transactional` **internal call:** *"It works through a proxy. An internal `this.method()` call bypasses the proxy, so no transaction starts."*
5. **Constructor injection:** *"Fields can be final, dependencies cannot be missing, and I can test with `new Service(mock)` without a Spring context."*
6. **Kafka vs RabbitMQ:** *"RabbitMQ is a postbox — delivered to a worker, then gone. Kafka is a ledger — events are retained and many consumer groups read independently. I used RabbitMQ for email tasks and Kafka for state-change events with multiple consumers."*
7. **Circuit breaker:** *"Closed, open, half-open. Failing fast stops the caller exhausting its threads and gives the failing service room to recover."*
8. **Index cost:** *"Faster reads, but every insert and update maintains every index, so I add them deliberately, not by default."*

Containers and CI/CD ●● — your biggest gap, named 3× in the JD

Say the boundary first: *"I have not owned the container and deployment side — that was a separate team. So, understanding rather than hands-on."* **Then explain it properly.**

- **Container vs VM** — a VM carries a whole guest OS; a container shares the host kernel and packages only the app and its dependencies. MBs and seconds, not GBs and minutes. VMs isolate more strongly.
- **Dockerfile → image → container** = recipe → cake → slice. **One image, many containers.**
- **Layer caching** — copy `pom.xml` and download dependencies *before* copying `src`, so a code change does not re-download everything. **Multi-stage build** throws away Maven and the JDK so the shipped image carries only the JRE and the JAR.

- **Config comes from env vars**, never baked into the image — same artefact everywhere, only configuration changes. Never put secrets in an image.
- **Pod** disposable, new IP each time · **Deployment** gives rolling updates and rollback · **Service** = a stable name and load balancing in front of changing pods — *"the role Eureka played in my Classifieds project"*.
- 🔴 **Readiness vs liveness** — ready for traffic yet, versus still alive. Confusing them is a real incident: a slow Spring Boot startup failing a *liveness* probe gets killed and restarted forever.
- **Rolling updates mean two versions run at once** → database migrations must be backwards compatible. Add a nullable column, deploy, then use it. **Expand then contract.**
- **CrashLoopBackOff** → `kubectl logs --previous`; usually a missing env var or a DB it cannot reach. **ImagePullBackOff** → wrong tag or registry credentials.
- **CI/CD:** build → test → quality gate → package → deploy. 🔴 **Build once, promote the same artefact** through SIT, UAT, prod — rebuilding per environment means you test one thing and ship another. Jenkins / GitHub Actions / Azure DevOps all express this same shape.
- **In BFSI the production step is gated and approved**, not automatic. Say that.

OAuth2 vs JWT 🔴

*"Not alternatives. **OAuth2 is how a token is obtained; JWT is a format a token can be written in.**"* Four roles: resource owner, client, authorization server, resource server. **Authorization Code flow** — the app never sees the password; the code is exchanged server to server so the token never sits in a URL. **PKCE** for mobile and SPAs. **Client credentials** for service-to-service. **OIDC is an identity layer on top of OAuth2** — OAuth2 authorises, OIDC authenticates. *Your honest line: you issued JWTs from your own service rather than integrating a full authorization server.*

ActiveMQ = a JMS broker, conceptually RabbitMQ not Kafka — queue or topic, delivered then gone. Still common in older banking estates, which is why it is on the JD.

Core Java

- **Overloading** = compile time, different parameters. **Overriding** = runtime, same signature, subclass. You cannot override a static method — it is hidden.
- **Abstract class** = shared state and code, one only. **Interface** = capability contract, many.
- **ArrayList** = array, O(1) get. **LinkedList** = nodes, O(n) get. Default to ArrayList.
- **HashMap** not thread safe · **Hashtable** locks the whole map, legacy · **ConcurrentHashMap** locks per bucket.
- **String immutable** because of the string pool, security, thread safety, cached hash.
- `==` compares references, `.equals()` compares values. `new String("a") != "a"`.
- **Checked** = compiler forces handling. **Unchecked** extends `RuntimeException`.
- `finally` always runs except on `System.exit()`. Never `return` from `finally`.
- **Fail-fast** throws `ConcurrentModificationException`; remove via `iterator.remove()`.

Java 8

- **Intermediate** ops are lazy (`filter`, `map`, `sorted`); **terminal** ops trigger (`collect`, `reduce`, `forEach`, `findFirst`).
- `map` = one to one. `flatMap` = one to many, then flattened.
- `Predicate` →filter · `Function` →map · `Consumer` →forEach · `Supplier` →lazy value.
- `groupingBy(X, counting())` and `groupingBy(X, summingDouble(Y))`.
- **Optional:** use `map` / `orElse` / `orElseThrow`, not `isPresent` + `get`.
  `orElse` always evaluates; `orElseGet` is lazy.

Spring Boot

- **IoC** = the framework controls creation. **DI** = how it hands dependencies in.
- Beans are **singleton by default**, so keep them **stateless** — a mutable instance field is a concurrency bug.
- `@Repository` also **translates database exceptions** into Spring's hierarchy.
- `@RestControllerAdvice` + `@ExceptionHandler` = one consistent error shape. Log the stack trace, return a generic message.
- **Auto-configuration** works through `@ConditionalOnClass` and `@ConditionalOnMissingBean` — your own bean always wins.
- `@WebMvcTest` = slice, fast · `@SpringBootTest` = full context · `@MockBean` = replace a bean.

JPA and Hibernate

- **JPA** = spec · **Hibernate** = implementation · **Spring Data JPA** = the layer on top.
- Defaults: `@ManyToOne` and `@OneToOne` are **EAGER**; `@OneToMany` and `@ManyToMany` are **LAZY**. Set ManyToOne to LAZY explicitly.
- `@Transactional` rolls back on **unchecked** exceptions only, unless `rollbackFor`.
- **Dirty checking** means a managed entity needs no `save()`.
- First-level cache is always on, per transaction. Second-level is opt-in and shared.
- Use `@Enumerated(EnumType.STRING)`, never the ORDINAL default.
- Money is **BigDecimal**, never `double`.

Microservices • *(mandatory in Posting B — and all over your resume)*

- Gains: independent deploy, independent scale, fault isolation. **Costs: network failure, no distributed transaction, harder debugging.** Say the costs.
- **Eureka** = discovery · **API Gateway** = single entry, auth and routing in one place · **Config Server** = externalised config.
- **Saga** = local transactions plus **compensating** transactions. Eventual consistency.
- Only retry **idempotent** operations. Always set a **timeout**.

- Split by **business capability**, not technical layer.
- Observability: centralised logs, **correlation ID** tracing, metrics.

Messaging

- Kafka: **topic → partitions → offsets**; order is guaranteed **within a partition**, so key by transaction or account ID.
- **Same consumer group** = share the partitions. **Different groups** = each gets everything.
- **At-least-once** is the normal default, so consumers must be **idempotent**.
- RabbitMQ: producer → **exchange** → queue. Direct, topic, fanout. **DLQ** after N failed attempts.

Security

- **JWT** = header.payload.signature. **Signed, not encrypted** — anyone can read the payload, so put nothing sensitive in it.
- Stateless, so any instance validates it — that is why it suits microservices.
- Cannot easily revoke: short-lived access token plus refresh token, or a blocklist.
- CSRF is off for a stateless API because the browser does not auto-attach an `Authorization` header.
- **BCrypt** for passwords — slow and salted by design.
- `hasRole("ADMIN")` looks for the authority `ROLE_ADMIN`.

SQL

- **Left-most prefix rule:** an index on `(status, created_at)` does not serve `created_at` alone.
- Index killers: a **function on the column**, and a **leading** `%` in LIKE.
- `EXPLAIN`: `type: ALL` means full scan · check `key`, `rows`, and `Extra`.
- `WHERE` filters rows **before** grouping; `HAVING` filters groups **after**.
- `UNION` deduplicates and sorts; `UNION ALL` does not and is faster.
- Customers with no transactions = `LEFT JOIN ... WHERE t.id IS NULL`.
- Deep `OFFSET` is slow — prefer **keyset pagination**.

Coding round

- Talk while you type. Ask about edge cases **first**. State the complexity at the end.
- `LinkedHashMap` for first-non-repeating. `seen.add(n)` returns false for a duplicate.
- Two sum = HashMap, O(n), trading space for time. Mention the brute force first.
- `Integer` cache is −128 to 127, so `128 == 128` is **false** for boxed Integers.

Delivery half of the JD ● — most candidates prepare none of this

- **Code review, what you look for, in order:** correctness and edge cases · error handling (nothing swallowed silently) · **the data layer** — N+1 in a loop, missing index, `@Transactional` on the right method · meaningful tests · readability last.
- **Giving feedback:** separate must-change from suggestion and say which · ask rather than instruct · past three replies, go and talk to them.
- **Unit test plan review:** cover **behaviour and rules**, not lines. The valuable tests are the **negative** ones — an approved request cannot be approved twice, a requester cannot approve their own. *(You wrote exactly these.)*
- **Coverage:** tells you what ran, not what was verified. A test with no assertion still counts.
- **Estimation:** break it down until you can picture doing each piece · include testing, review turnaround and **integration** · **state the assumptions**, so a change is a re-estimate rather than a slip.
- **Technical risk planning:** what could go wrong, how likely, how bad, what we do about it. *Your example: moving email to RabbitMQ risked silent failure, so retries with backoff, a DLQ, and alerting on the DLQ. Async without a DLQ moves the failure somewhere nobody is looking.*
- **Design review:** does it meet the requirement · what happens when a dependency is down · single points of failure · authorisation server side · how would we know it works in production · how do we roll back · **is it more complex than the problem needs**.
- **Client raises a quality issue:** acknowledge fast · contain first, cause second · do not commit to a cause before you know, do commit to when you will update · fix, verify, say what changed so it does not recur. *"Being wrong quickly and openly costs far less than being confident and wrong slowly."*
- **Leading a team:** be honest — you have not led one. You have reviewed code and paired. Explaining *why* sticks; pointing at the answer does not.
- **Knowledge management:** Swagger docs that live with the code · runbooks · handover notes. Test = could someone pick up my work if I were away.
- **High quality code:** correct · readable · tested where it matters · performs at real data volumes (both your fixes were fine in testing and slow in production) · **traceable** — logging and audit trail, which in a bank is part of the deliverable.

Your stories — the numbers

15s → 6s report: N+1 found via SQL logging, fixed with a projection query, indexes, rewritten JOINs. *"My first guess was wrong, which is why I measure first."*
2–3s → under 1s APIs: N+1 plus lazy loading plus indexing.
RabbitMQ: email off the request thread, bulk approvals stopped timing out.
Kafka: state changes to multiple consumer groups, retained and replayable.
RBAC: requester, approver, admin — enforced server side, because a UI check only hides the button.
Classifieds: 3 services, own databases, Eureka, API Gateway, JWT, global exception handling.

The three questions you must not fumble

"Tell me about yourself" → Now → What you built → Why this role. Under 2 minutes.

"Walk me through your timeline." → B.Tech 2022 · JSPIDERS Java programme May 2023 – Jan 2024 · Artifint technical recruitment Feb – Dec 2024 · Zensar from Dec 2024. The one gap is 2022 to mid-2023 — two sentences, the **real** reason, no apology.

"Why recruitment, then development?" → B.Tech in CS, development was always the plan, JSPIDERS to build the skills, and the Artifint role was **technical** recruitment so I was close to the work. **Then turn it:** *"That year is why I am comfortable in client and requirement conversations"* — which is the first responsibility listed in both postings.

The three answers to rehearse most *(more than any technical one)*:

Experience band (JD says 3–6) → *"In engineering, December 2024 to June 2026 — about a year and a half — and around two and a half years professional overall. I know that is below the range."* Then straight to evidence: UK bank platforms, **15s → 6s**, **2–3s → under 1s**, Kafka and RabbitMQ, production incidents. **Never round up — they verify against payslips.**

The gap (you left in June) → two sentences, the real reason, then **specifically** what you did since. *"I have been working through containerisation and deployment, which was the part of my last role I was furthest from"* is honest and it is exactly what this JD asks for. Close on being available immediately.

Recruitment → development → B.Tech CS, JSPIDERS, and Artifint was **technical** recruitment. Then turn it: *"that year is why I am comfortable in client and requirement conversations"* — and this is a **client-facing GCC role**.

"How much experience do you have?" → Answer with dates, never a rounded number. *"Engineering since December 2024, so about a year and nine months, plus a year in technical recruitment before that."*

Questions to ask them

"Which domain would this role sit in?" · "Greenfield microservices or modernising a monolith?" · "What would a successful first three months look like?" ·
"What is the biggest technical problem this hire would help with?"

The last five minutes

- You have real production numbers and banking domain experience. Most candidates for this role have neither.
- **If you do not know something: say so, then reason out loud.** That has never cost anyone an offer. Bluffing has.
- Give the **why**, not just the **what**. "HashMap stores key-value pairs" is a textbook answer; "O(1) because it hashes to a bucket instead of scanning" is an engineer's.

- Sit up. Breathe out slowly. Smile before you speak — it changes your voice.

Go get it.